Université Marie et Louis Pasteur (UMLP)

Master of Science in Advanced Robotics, Mechatronics, and Automatic Control (ARMAC)

Master Thesis

Multimodal Language-Grounded Frameworks for Autonomous Robot Navigation and Interaction

Author:
Promise Emeziem Adiole

Supervisors:
Prof. Elmar Rueckert
Nwankwo Linus (PhD)

Host Institution:
Cyber-Physical Systems Group,
Montanuniversität Leoben, Austria

Academic Year:
2024–2025

Abstract

This thesis presents a language-grounded autonomous navigation system that integrates large language models and vision–language models with the ROS 2 Navigation 2 framework, and — more consequentially — an evaluation methodology that measures what such a system achieves rather than what it reports.

The system accepts open-ended natural-language commands by speech or text, parses them into structured intents, grounds place names against a metric registry, executes them through Navigation 2 on a differential-drive ROMR mobile robot, and perceives its surroundings in open vocabulary by composing the Segment Anything Model with CLIP. The command policy is an explicit LangGraph state machine with dedicated verification and recovery states, instrumented so that every node records its timing, its branch and the reason for that branch. **All experimental work is conducted in Gazebo simulation**, using a model of a physical robot held in the laboratory; deployment onto that robot was attempted but its on-board Jetson Nano could not host the segmentation and image–text models.

Arrival is scored against the simulator's own pose for the robot, obtained through a channel that shares no sensor, no filter and no coordinate frame with the robot's localisation estimate. Reported success and actual arrival are therefore measured independently and tabulated separately.

Language understanding was not the limiting factor: 60 of 61 commands resolved to the intended destination across eight destinations and deliberately varied phrasing. Navigation was. In the initial evaluation the stack reported success on 12 of 16 navigation trials while only 2 of those 12 had physically arrived within 0.5 m. **Ten of twelve reported successes — 83 % — were false**, with a median true error of 6.35 m and a maximum of 15.66 m.

The ordinary explanations were tested and eliminated. All eight destinations lie in a single connected traversable region with at least 1.09 m of clearance, and a scan taken at the true pose places 100 % of beams within 0.20 m of a mapped wall, mean 4.7 cm, with a single sharp minimum at the correct heading. The environment is informative and the goals are reachable. Isolating the responsible motion showed that standing still and driving straight cost nothing, while one 90° rotation multiplied heading uncertainty by a factor of 9.3.

Diagnosis identified three faults in the particle filter — a sensor range cutoff of half the fitted scanner's reach, a rotational noise coefficient an order of magnitude too large, and a hit-versus-noise weighting that discounted a demonstrably reliable scan — together with a fourth fault in which navigation outcomes carried no identity of the goal that produced them. A parameter sweep established that the intuitively correct correction, admitting *more* rotational noise to account for the robot's scrubbing casters, moved the system in the wrong direction; the relationship was monotonic the other way.

Re-running the identical protocol after correction gave 60 arrivals in 61 commands (98 %), no false positives among 50 reported successes, a median true error of 0.231 m, and median

belief-to-truth drift of 0.108 m against 6.25 m before. The residual discrepancy has inverted: the stack now under-reports, leaving ten genuine arrivals unconfirmed.

The contribution is threefold: an integrated, fully traceable language-grounded navigation system; evidence that evaluations scoring task success against the estimator under test can overstate performance by a factor of six; and a worked demonstration that the failure so exposed was a configuration fault, removable once measured. The remedy proposed is inexpensive and immediate — measure against something the robot cannot see, and publish both figures.

Acknowledgements

This research was carried out as part of the Master's programme in Advanced Robotics, Mechatronics, and Automatic Control (ARMAC) at Université Marie et Louis Pasteur. I wish to express sincere gratitude to Professor Elmar Rueckert for his invaluable supervision, insightful feedback, and intellectual guidance throughout the internship and thesis development. Special thanks are also extended to Nwankwo Linus for his mentorship and constructive discussions that shaped the direction of this work.

I gratefully acknowledge the support of the Cyber-Physical Systems Group at Montanuniversität Leoben, whose collaborative environment and technical resources made this research possible. Appreciation is also due to colleagues and peers in the ARMAC programme for their assistance, encouragement, and meaningful exchange of ideas.

Finally, heartfelt thanks are offered to my family and friends for their unwavering support, patience, and inspiration during the entire course of study and research.

AI Assistance Disclosure

The author utilized AI technology, specifically "ChatGPT", as a writing assistant to refine the clarity, coherence, and style of the thesis report. All conceptual contributions, scientific analyses, system implementations, and research decisions remain the sole responsibility of the author.

Contents

1 Introduction

1.1 Background of the Topic

A mobile robot that can be told what to do in ordinary language has been a standing objective of autonomous systems research since the earliest service-robotics programmes. Until recently the obstacle was representational: a robot could hold a metric map and a set of hand-authored behaviours, but it had no way to connect the word *kitchen* to a region of that map unless a programmer had written the connection down in advance. Every new instruction required new code, and every new environment required the vocabulary to be rebuilt from nothing.

Large language models and vision–language models changed the terms of that problem. A language model can take an open-ended instruction and emit a structured intent without the instruction having been anticipated. A vision–language model such as CLIP (Radford et al., 2021) can decide whether an image region matches an arbitrary phrase, and a class-agnostic segmenter such as the Segment Anything Model (Kirillov et al., 2023) can propose those regions without a fixed label set. Composing the two yields open-vocabulary perception: the robot can be asked about objects nobody enumerated at design time.

What these models do not supply is embodiment. A language model that emits `NAVIGATE_TO_PLACE(kitchen)` has produced a symbol, not a motion. The symbol must be resolved to a metric pose, the pose dispatched to a motion planner, the plan executed against real actuators, and the outcome observed. Each of those steps belongs to a mature and largely separate discipline – probabilistic localisation (Thrun et al., 2005), graph-search and sampling-based planning, and closed-loop control – and each carries failure modes that the language layer above it can neither see nor reason about. This thesis is concerned with the seam between the two: what happens when a fluent, competent language interface is bolted to a conventional navigation stack, and what it takes to find out whether the resulting system does what it says it does.

1.2 Research Problem

The research problem is one of *unobservable failure*. In a layered autonomy stack, each layer reports its outcome to the layer above. The language layer asks the navigation stack whether the goal was reached; the navigation stack answers from its own estimate of where the robot is. If that estimate is wrong, the answer is wrong, and nothing above the estimate is in a position to notice. The robot then reports a successful arrival, in fluent natural language, at a place it is not.

This is a different failure mode from the ones the language-grounding literature usually addresses. It is not a misunderstanding of the instruction, nor an error of semantic grounding: the language layer may be operating perfectly. It is a failure of the system's self-knowledge, and it is invisible to every conventional evaluation, because conventional evaluation asks the system to grade itself.

The problem this thesis addresses is therefore twofold. First, how can a language-grounded navigation system be evaluated against a source of truth it does not control, so that confidently wrong self-reports are detectable rather than counted as successes? Second, once such failures are exposed, what are their actual causes, and can they be removed?

1.3 Research Gap

Three gaps in the existing literature motivate this work; they are developed in detail in Chapter 2 and stated here in summary.

Evaluation against self-report. Language-conditioned navigation benchmarks predominantly score success using the agent's own or the simulator's task-completion flag as mediated by the agent's pose estimate. Where a simulator provides ground truth it is often used to define the goal region but not to audit the agent's belief. The consequence is that localisation error is absorbed into the reported success rate rather than being separated from it, and a system whose language layer is sound cannot be distinguished from one whose localisation is sound.

Composed systems reported as monoliths. Work that composes an LLM planner with an off-the-shelf navigation stack typically reports end-to-end success without attributing failures to a layer. When a command fails it is rarely established whether the intent was misparsed, the place misresolved, the plan infeasible, or the robot simply mislocalised. Without attribution, the reported number tells a practitioner nothing about what to fix.

Negative results and their resolution. Where poor performance is reported it is usually treated as a limitation to be noted rather than a diagnostic signal to be followed. The path from a measured failure, through the identification of its parameter-level cause, to a re-run of the same protocol demonstrating the repair, is seldom shown end to end.

This thesis addresses all three: it builds an independent ground-truth protocol, attributes failure to a specific layer and a specific set of parameters, and re-runs the identical protocol after repair.

1.4 Research Questions and Objectives

The work is organised around four research questions.

RQ1 Can an open-ended natural-language instruction be resolved to a correct navigation goal reliably enough that language understanding is not the limiting factor in a language-grounded mobile robot?

RQ2 What is the magnitude of the discrepancy between the arrivals such a system reports and the arrivals that physically occur, when the two are measured independently?

RQ3 To what layer, and to what specific mechanism, is that discrepancy attributable?

RQ4 Once the mechanism is identified and corrected, does the same evaluation protocol show the discrepancy removed?

The corresponding objectives are:

1. To design and implement a full language-to-motion pipeline: speech and text input, intent parsing, open-vocabulary perception, place resolution, and dispatch to a standard ROS 2 navigation stack, exposed through a web interface that makes every stage inspectable.
2. To construct an evaluation harness that scores arrival against the simulator's own pose for the robot, sharing no sensor, filter, or coordinate frame with the robot's estimate.
3. To run a controlled suite of natural-language commands through the complete system and report reported-versus-actual arrival separately.
4. To diagnose the dominant failure mechanism at parameter level and verify the correction by re-running the identical protocol.

1.5 Why the Study Is Important

The practical importance of this work is that a confidently wrong self-report is a more dangerous failure than an acknowledged one. A robot that reports uncertainty can be supervised; a robot that reports success from the wrong room cannot, because nothing in its output distinguishes the two cases. As language interfaces move from demonstrations into settings where a person acts on the robot's word – a delivery confirmed, an inspection reported complete, a part placed in a shared workspace – the gap between reported and actual becomes a safety property rather than a performance metric.

The methodological importance is that the measurement is cheap and the correction was tractable. Ground truth in simulation costs one process call per trial. The audit it enables changed the headline result of this project from a 22% success rate to a 6% arrival rate, and then, once the cause was found, to 98%. None of those three numbers could have been obtained by asking the system how it was doing.

Finally, the work contributes a worked negative result and its resolution. The most instructive finding reported here is that the intuitively correct correction to the localisation filter made the system measurably worse, and that only a parameter sweep scored against ground truth revealed that the sign of the adjustment was inverted. That is an argument for the method, not merely a result produced by it.

1.6 Scope and Limits of the Study

The study is bounded as follows.

Simulation. All quantitative results are obtained in Gazebo Classic 11. The physical robot exists and hardware deployment was attempted, but the on-board Jetson Nano could not host the segmentation and image–text models; that limitation is itself reported in Section 6.5.1 and Chapter 6 rather than omitted.

One platform, one environment. A single differential-drive robot in a single indoor environment of 13.9×11.6 m, with eight recorded named places. No claim is made about multi-robot operation, outdoor operation, or unmapped exploration.

Navigation, not manipulation. The action space is planar motion to a pose. Grasping, assembly, and force-controlled interaction are outside scope.

Perception is used for description, not for goal derivation. The vision–language pipeline reports what the robot can see and answers questions about it. Navigation goals are resolved from a recorded place registry rather than from visual search, so a failure of perception degrades the robot's description of its surroundings but does not by itself send it to the wrong place.

No human-subject data. The command suite is author-constructed, and the ethical considerations that follow from that choice are set out in Section 3.8.

1.7 Overview of Each Chapter

Chapter 2 reviews classical geometric navigation, transformer representations, vision–language models, language models as embodied planners, language-conditioned navigation, retrieval-augmented generation, and the reported failure modes of language-grounded autonomy. It establishes where the literature agrees, where it disagrees, and what is missing, and states the theoretical framework adopted here.

Chapter 3 states the research design, the instruments, and the system built to answer the research questions: the robot and its sensing, the simulated environment and map, the perception and language stack, the LangGraph command policy, the navigation configuration, the evaluation harness, the data-collection procedure, the analysis method, and the ethical considerations.

Chapter 4 reports the findings, organised by research question, with tables and figures and without interpretation.

Chapter 5 interprets those findings, compares them with previous studies, accounts for the similarities and differences, and argues their importance.

Chapter 6 summarises the main findings, states the conclusions and contributions, gives practical recommendations, sets out the limitations honestly, and proposes future research.

1.8 Publications Resulting from the Thesis

No peer-reviewed publications have resulted from this thesis at the time of submission. This is stated plainly rather than deferred: the work has been conducted and documented as a single-author project, and the results have not yet been submitted for external review.

The validation study reported in Chapters 4 and 5 – specifically the measurement of the gap between reported and actual arrival, its attribution to the localisation filter, and the demonstration that the identical protocol shows the gap closed after correction – is the component the author considers publishable, and is intended for submission to a robotics evaluation or field-robotics venue. The complete implementation, the command suites, and the per-trial data supporting every figure in this thesis are available in the accompanying public repository, so that the results can be reproduced rather than taken on assertion.

2 Literature Review

This chapter reviews the work this thesis builds on, identifies where the field agrees and where it does not, isolates what is missing, states how this study differs, and sets out the theoretical framework adopted.

2.1 Summary of Previous Studies

2.1.1 Probabilistic Localisation and Classical Navigation

The dominant formulation of indoor mobile-robot localisation remains recursive Bayesian estimation over a known map. Monte Carlo Localisation (Thrun et al., 2005) represents the pose posterior as a weighted particle set, propagating particles through a motion model and reweighting them under a sensor model; adaptive variants adjust the sample count to the concentration of the posterior. The standard treatment (Thrun et al., 2005) establishes both the algorithm and its known pathologies: particle depletion, sensitivity to an over-confident motion model, and the inability to recover from divergence without deliberate reinjection.

Planning and control are similarly settled. Global planning over an occupancy grid using a wavefront or graph-search method, combined with a local sampling controller such as the Dynamic Window Approach (Fox et al., 1997), is the standard composition, and is what the Navigation 2 stack (Macenski et al., 2020) packages for ROS 2. Critically for this thesis, this entire pipeline is conditioned on the pose estimate. The planner plans from where the filter says the robot is; the goal checker declares arrival when the filter says the robot is close enough. Localisation error therefore propagates into every downstream decision without being visible in any of them.

2.1.2 Representation Learning and Vision–Language Models

The perception subsystem composes two pretrained models. Neither is trained or adapted in this work, so what follows treats only the properties the composition relies on, and the quantities it produces that are reported in Chapter 4.

Contrastive Image–Text Pretraining

CLIP (Radford et al., 2021) trains an image encoder f_I and a text encoder f_T to map their inputs into one d-dimensional space, so that an image and a phrase describing it land close together. Embeddings are ℓ_2-normalised, which places them on the unit sphere and reduces similarity to a dot product:

$$s(I,T) \;=\; \frac{\langle f_I(I),\, f_T(T)\rangle}{\|f_I(I)\|\,\|f_T(T)\|} \;\in\; [-1,1]. \tag{2.1}$$

Training maximises this similarity for matched pairs and minimises it for mismatched ones. Over a batch of N pairs, with $s_{ij} = s(I_i, T_j)$ and a learned temperature τ, the objective is symmetric in the two directions:

$$\mathcal{L} = -\frac{1}{2N}\sum_{i=1}^{N}\left[\log\frac{e^{s_{ii}/\tau}}{\sum_j e^{s_{ij}/\tau}} + \log\frac{e^{s_{ii}/\tau}}{\sum_j e^{s_{ji}/\tau}}\right]. \tag{2.2}$$

The consequence that matters here is that no label set is fixed at training time. Recognition becomes retrieval: given a candidate vocabulary $\mathcal{V} = \{T_1, \ldots, T_M\}$ chosen at run time, a region is labelled by

$$p(T_k \mid I) = \frac{e^{s(I,T_k)/\tau}}{\sum_{m=1}^{M} e^{s(I,T_m)/\tau}}, \qquad \hat{\ell}(I) = \arg\max_k \; p(T_k \mid I). \tag{2.3}$$

Equation (2.3) is the origin of the confidence values reported in Section 4.6, and it carries a caveat that should be read with them. The softmax is taken over the vocabulary offered, so the score is the model's preference *among the candidates supplied*, not a calibrated probability that the object is present. Removing a competing phrase raises the score of the remainder without any change to the image. A value of 0.70 therefore means that phrase dominated the alternatives it was scored against, and nothing stronger.

Figure 2.1 shows the two phases together. The left panel is Equation (2.2): a batch of image–text pairs scored against each other, with the matched diagonal driven up and everything off it driven down. The right panels are Equation (2.3), where a vocabulary supplied at run time is embedded once and each image labelled by its nearest phrase. It is this second construction the perception subsystem uses, with the vocabulary chosen for the environment rather than for a benchmark.

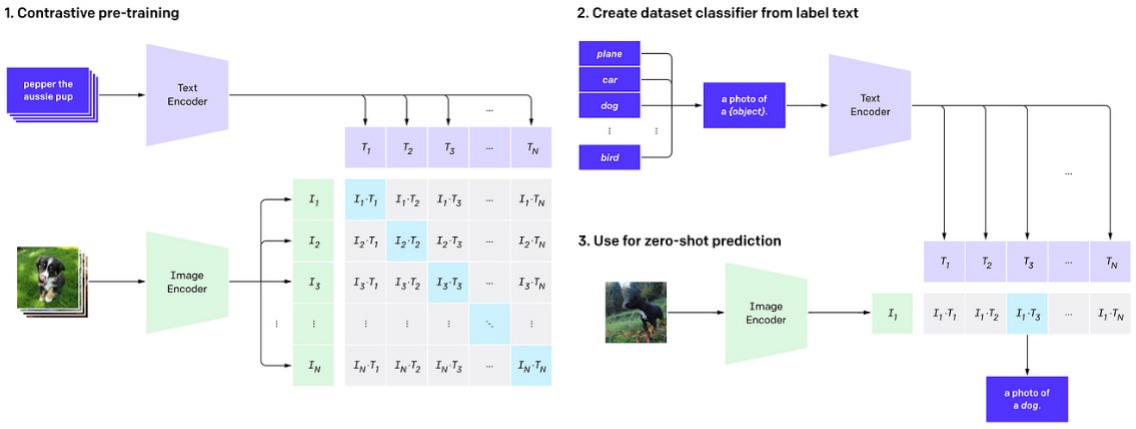

Figure 2.1. Contrastive pretraining and zero-shot inference. Reproduced from Radford et al. (2021). Left, the training objective of Equation (2.2). Right, the inference construction of Equation (2.3): candidate phrases are embedded once and an image is labelled by the closest, so the label set is fixed at run time rather than at training time.

Class-Agnostic Segmentation

CLIP compares whole images to phrases; it does not say which part of an image to compare. The Segment Anything Model (Kirillov et al., 2023) supplies that, in three parts: a heavyweight image encoder producing a dense embedding, a lightweight prompt encoder taking points, boxes or coarse masks, and a mask decoder combining the two to emit candidate masks with predicted quality scores.

The architectural property the composition depends on is the asymmetry between those parts. The image embedding is computed once per frame; the decoder is cheap enough to run many times against it. Prompting on a regular grid of points therefore yields a full set of region proposals for roughly the cost of one encoder pass. Crucially the masks carry no labels — the model proposes *what* is a coherent region, never *what it is*.

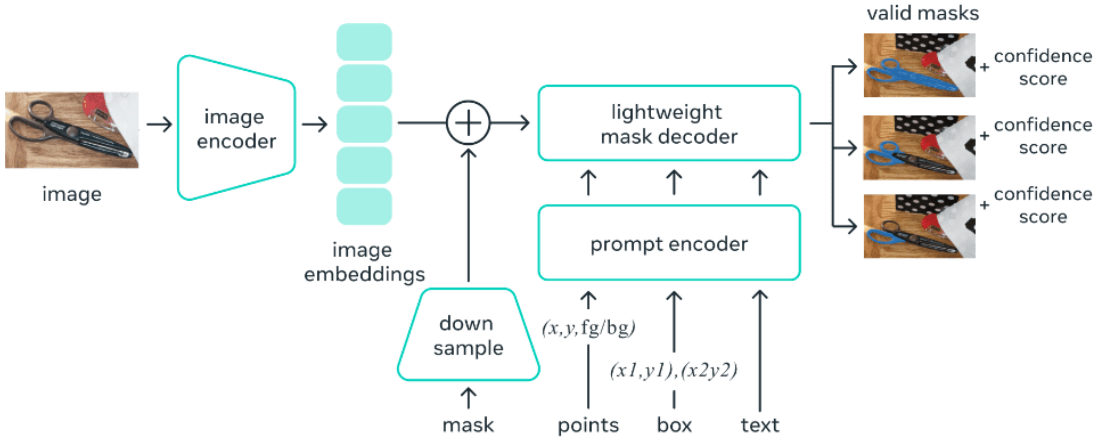

Figure 2.2. The Segment Anything model structure. Reproduced from Kirillov et al. (2023). The image encoder is run once per frame to produce a dense embedding; the prompt encoder and mask decoder are light enough to be run repeatedly against it, which is what makes prompting on a grid of points affordable. The masks carry confidence scores but no class labels, and it is that omission the composition of Section 2.1.2 fills.

The Composition Used Here

The two supply complementary halves, and the pipeline is their concatenation. For a frame I, SAM proposes masks $\{m_1, \ldots, m_J\}$; each is cropped to its bounding box to give a region I_j; and each region is labelled by Equation (2.3) against the run-time vocabulary. The result is open-vocabulary detection without a detector ever having been trained on the classes involved.

The variants used are the smaller released configurations — `vit_b` for SAM, and a ViT-B backbone with 32×32 patches for CLIP, whose coarser patching yields fewer tokens and a correspondingly cheaper forward pass. That choice is a compute decision rather than an accuracy one, and it did not prove sufficient: the pipeline could not be hosted on the robot's on-board computer, as reported in Section 6.5.1.

Captioning models such as BLIP-2 (Li et al., 2023) offer a third capability, free-form description. That family was evaluated during this project but is not part of the delivered system, for reasons given in Section 3.4.3.

2.1.3 Language Models as Planners

A substantial body of work treats a language model as a task planner that emits executable structure rather than prose. Code-as-Policies (Liang et al., 2023) has the model emit executable policy code; ProgPrompt (Singh et al., 2023) casts planning as program synthesis against an API of available skills; SayCan (Ahn et al., 2022) weights proposed actions by a learned estimate of whether the robot can actually perform them in the current state. The common thread is that the language model is constrained to a grounded action space rather than allowed to free-associate.

2.1.4 Language-Conditioned Navigation

Vision-and-language navigation formulates the problem as following an instruction through a previously unseen environment (Anderson, Wu et al., 2018). The Room-to-Room formulation supplies the convention the field still largely works within: an agent moves between nodes of a navigation graph built over a scanned building, and a trial counts as a success when the agent stops within a fixed radius of the goal. Because the graph is given, motion is a discrete transition rather than an executed trajectory.

Evaluation in this line was formalised shortly afterwards (Anderson, Chang et al., 2018), which argued that a bare success rate rewards exhaustive search and proposed weighting each success by the ratio of the shortest path to the path actually taken. That measure, success weighted by path length, remains the standard alongside success rate and final distance to goal. It is worth being precise about what it addresses: it penalises an inefficient route, and presumes throughout that the agent's position is known.

The nav-graph assumption was subsequently relaxed (Krantz et al., 2020), which reformulated the task in continuous environments where the agent issues low-level actions instead of selecting graph nodes, and reported that performance degrades markedly once execution is required rather than assumed. That result is the closest antecedent to the argument of this thesis: it establishes that the gap between deciding where to go and getting there is large enough to dominate a reported score.

A second line replaces the fixed goal set with a spatial representation that can be queried in language. Visual language maps (Huang et al., 2023) fuse vision–language embeddings into a metric map so that a phrase resolves to coordinates; related work indexes open-vocabulary detections into a queryable scene representation (Chen et al., 2023) or drives object-goal navigation directly from an image–text scorer without task-specific training (Gadre et al., 2023). Systems in this family have been assembled on physical platforms: LM-Nav (Shah et al., 2023) composes a language model, an image–text model and a navigation policy to follow instructions outdoors without fine-tuning any of the three.

Work in the ReLI line (Nwankwo et al., 2025) and on conversational robot interfaces (Nwankwo & Rueckert, 2024) addresses the interaction layer directly, treating the dialogue as part of the control loop rather than as a front end.

One feature is common to the benchmark settings above and is central to what follows. Whether the agent selects graph nodes or issues velocities, its position is supplied by the simulator. It is not estimated by the agent from its own sensors, and cannot therefore be wrong in the way a running localisation filter can be wrong. Localisation error is consequently absent from the error budget these benchmarks account for — not overlooked, but outside the formulation. Section 2.3 returns to what that excludes.

2.1.5 Retrieval-Augmented Generation

Retrieval-augmented generation (Lewis et al., 2020) conditions a language model on passages retrieved from an external corpus, reducing unsupported generation and allowing the knowledge base to be updated without retraining. Evaluation frameworks for such systems (Es et al., 2024) score faithfulness and answer relevance without reference answers. In an embodied setting the same machinery lets a robot answer questions about its own configuration and history from documents rather than from parametric memory.

2.2 Where Researchers Agree and Disagree

There is broad agreement on three points. First, that grounding is the binding constraint: a language model must be tied to an action space the robot can actually execute, and the mechanisms proposed for this – affordance weighting, program synthesis against a skill API, structured intent – differ in form but agree in purpose. Second, that open-vocabulary perception is best obtained compositionally, by pairing a region proposer with an image–text scorer, rather than by training a closed detector. Third, that natural language is an appropriate interface for non-expert operation of a mobile robot.

There is much less agreement on evaluation. Success criteria vary between distance-to-goal thresholds, simulator completion flags, task-specific predicates, and human judgement, and the same nominal success rate can mean materially different things across two papers. More consequentially for this thesis, the field disagrees implicitly about *whose* account of the outcome counts. Some benchmarks derive success from a simulator oracle; others accept the agent's own determination. Where the agent's pose estimate mediates that determination, localisation error is silently folded into the reported score.

A second disagreement concerns where responsibility for robustness lies. One position holds that the language layer should reason about uncertainty and verify its own actions; another holds that the navigation stack should expose honest confidence and the language layer should simply relay it. This thesis takes the second position and implements it explicitly, for reasons argued in Section 2.5.

2.3 What Is Missing in Past Research

Three specific omissions follow from the review above.

Independent audit of the arrival claim. Where ground truth is available, it is commonly used to define the goal region rather than to interrogate the agent's belief about its own position. In the benchmark settings of Section 2.1 the question does not arise, because the agent's position is given; it arises as soon as a system estimates its own position and acts on that estimate, which is the setting of any deployed robot. The quantity of interest here – the distance between where the robot reports being and where it is – is rarely reported, and consequently the false-positive rate of reported successes is rarely known.

Failure attribution across layers. Composed systems are typically scored end to end. When a command fails, published results seldom separate misparsed intent, misresolved place, infeasible plan, and mislocalisation. This matters because the four have entirely different remedies, and because a system can post a poor end-to-end score while its language layer is flawless.

The full diagnostic arc. Negative results are reported; the traverse from a negative result to its parameter-level cause, and then to a re-run of the identical protocol demonstrating repair, is not commonly shown. Without that traverse it is difficult to distinguish a fundamental limitation from a misconfiguration.

2.4 How This Study Differs

This study differs in four respects.

First, arrival is scored against the simulator's own pose for the robot, obtained through a channel that shares no sensor, no filter and no coordinate frame with the robot's estimate. Reported success and actual arrival are tabulated separately, and the false-positive rate between them is reported as a first-class result.

Second, failure is attributed to a layer and then to a parameter. The thesis reports not that navigation was unreliable but that the particle filter's sensor model was configured for a shorter-range scanner than the one fitted, that its rotational noise term had the wrong magnitude, and that its hit-versus-noise weighting understated the informativeness of the scan.

Third, the correction is verified by re-running the identical command protocol rather than by argument, and both the before and after figures are reported from the same harness.

Fourth, the system is instrumented so that the reasoning is inspectable at run time. Every command traverses an explicit state machine whose route is recorded and surfaced in the operator interface, and the reply refuses to confirm an arrival when the filter's own covariance indicates the robot does not know where it is.

2.5 Theoretical Framework

The framework adopted here has two components.

2.5.1 Layered Autonomy with an Honest Interface

The system is a layered stack: interface, intent, policy, perception, middleware, navigation, simulation. The governing principle is that each layer must expose its own uncertainty to the layer above rather than assert a binary outcome. Concretely, the navigation layer does not answer "arrived"; it answers "the goal checker was satisfied, and the pose posterior had this covariance". The policy layer is then in a position to decline to confirm an arrival it cannot support. This is a deliberate rejection of the alternative position, in which the upper layer is expected to infer failure from behavioural cues, because a mislocalised robot produces no such cues – it behaves exactly as a correctly localised one would.

2.5.2 The Particle Representation of Belief

The localisation subsystem is a particle filter, and because the central finding of this thesis concerns what that filter believed and how sure it was, the representation itself is part of the framework rather than an implementation detail.

A particle filter represents the posterior over the robot's state as a set of weighted samples (Thrun et al., 2005):

$$\mathcal{X} \;=\; \left\{ \langle x^{[j]},\, w^{[j]} \rangle \right\}_{j=1,\ldots,J} \tag{2.4}$$

where each $x^{[j]}$ is a complete *state hypothesis* — one candidate answer to the question of where the robot is — and $w^{[j]}$ is its *importance weight*, the degree to which that hypothesis explains the most recent observation. The belief is then the weighted sum of point masses at those samples,

$$p(x) \;=\; \sum_{j=1}^{J} w^{[j]}\, \delta_{x^{[j]}}(x) \tag{2.5}$$

with $\delta_{x^{[j]}}$ the Dirac delta centred on particle j.

Two properties of this construction matter for the argument of this thesis.

The density carries the probability. Equation (2.5) places no functional form on the belief; the estimate of how probable a region is comes from how many samples fall inside it. A smooth curve drawn through the samples is a reconstruction, not something the filter stores. This is directly observable in the running system: because AMCL here resamples on every update, low-weight particles are discarded and high-weight ones duplicated, after which all weights are reset to uniform. Measured on this robot, every particle carried a weight of exactly $1/2000 = 5.00\times10^{-4}$, so between updates Equation (2.5) reduces to $p(x) = J^{-1} \sum_j \delta_{x^{[j]}}(x)$ and the weights convey nothing at all. The belief is the arrangement of the samples.

The filter as a recursive cycle. The estimate is maintained rather than recomputed. Each update has two steps: a *prediction*, which draws a new sample set from the proposal by pushing

every particle through the motion model, and a *correction*, which reweights those samples by the ratio of target to proposal. The belief at one instant is the input to the next, so the filter is a recursive Bayes filter whose distribution is carried non-parametrically by the samples themselves.

Two properties of that cycle are visible in the measurements of Chapter 4. The first is that *no motion means no update*. The cycle is triggered by odometry exceeding a threshold, so a stationary robot runs neither step and its estimate does not change — which is exactly what Table 4.5 records, where eight seconds at rest left position error at 0.015 m and heading spread unchanged to three decimal places. It is also why the filter publishes nothing while parked, a behaviour that reads as a fault and is not one (Section 3.5).

The second is that estimate quality improves with the number of samples, which is the justification for the adaptive count in Listing 3.2: the filter draws more particles when the posterior is diffuse and fewer when it has concentrated, spending computation where it reduces sampling error. That relationship holds only within the proposal's support, however. More samples drawn from a proposal centred in the wrong place estimate the wrong distribution more precisely, which is the formal statement of why the divergence reported in Section 4.4 was not something the filter could resolve by working harder.

Where the weights come from. The weights are not a modelling choice; they are what makes sampling from the wrong distribution admissible. The posterior cannot be sampled from directly, so samples are drawn from a *proposal* π that can be sampled — here, the previous particle set pushed through the motion model — and the discrepancy is carried by a weight

$$w(x) \;=\; \frac{f(x)}{\pi(x)}, \tag{2.6}$$

where f is the target. Expectations under f are then estimated from samples drawn under π and reweighted. In the localisation case the motion model supplies π and the terms cancel to leave the measurement likelihood, so $w^{[j]} \propto p(z_t \mid x_t^{[j]})$: a particle is weighted by how well the scan would be explained if the robot were there. The sensor-model parameters of Listing 3.2 are the parameters of that likelihood, which is why mis-stating them changes the estimate rather than merely its confidence.

Equation (2.6) carries a precondition that matters more here than the formula: the proposal must cover the target, $f(x) > 0 \Rightarrow \pi(x) > 0$. A region the proposal never proposes receives no samples, and no weighting can recover it, because there is nothing there to reweight. Two consequences follow for this thesis. A proposal spread too wide places most of its samples where the target has little mass, so the weights concentrate on a few particles and the effective sample size collapses — the mechanism behind the monotonic sweep of Section 4.4, where increasing the motion noise degraded the estimate rather than making the filter appropriately cautious. And once the estimate has diverged, the proposal is centred on the wrong place entirely; the true pose lies outside its support, and the filter cannot return unaided. That is why recovery

requires deliberate reinjection rather than more evidence, and why the divergence measured in Chapter 4 persisted until the pose was re-seeded.

One half is chosen, the other is forced. The asymmetry between the two distributions in Equation (2.6) is worth stating, because it separates two kinds of configuration error that are otherwise easy to conflate.

The proposal is a design choice. Any distribution that can be sampled from will do, provided it covers the target's support, and a different choice yields a different but equally valid filter. What the choice governs is efficiency: a proposal that places its samples where the target has mass produces nearly uniform weights and lets every particle contribute, while one that scatters them where the target is near zero concentrates the weight on a handful and leaves the rest doing no work.

The weight is not a choice. Once the proposal is fixed and the target is known, Equation (2.6) determines it; the weight exists precisely to undo the bias the proposal introduced. Substituting a different weight does not give a differently tuned estimate of the same posterior — it gives an estimate of a different distribution.

The consequence for configuration is that the two halves fail differently. A badly chosen proposal degrades the estimator while still targeting the right posterior. A mis-stated weight changes what is being estimated. The first is a tuning fault; the second is a false statement about the sensor, and is therefore checkable against a physical measurement. Both kinds were present in this system, and Section 4.4 separates them.

Resampling, and what it costs. A third step completes the cycle. After weighting, J particles are drawn from the existing set with replacement, particle i chosen with probability $w_t^{[i]}$. High-weight hypotheses are duplicated, low-weight ones discarded, and the weights are reset to uniform. This is why the weights measured on this robot were all exactly $1/2000$: with `resample_interval` set to 1 (Listing 3.2) the step runs on every update, so a subscriber essentially never observes the weighted set. What the weights expressed before resampling, particle *multiplicity* expresses after it, which is the precise sense in which the density carries the probability.

Resampling is also where diversity is lost. A particle discarded is not recovered, and repeated resampling drives the set toward copies of a shrinking number of ancestors — the particle depletion noted in Section 2.1. The only source of fresh diversity in this configuration is the noise injected by the motion model during prediction, which places the coefficients of Listing 3.2 under opposing pressures: too large and the proposal disperses faster than one correction can reconcile, too small and resampling starves the set of the variety it needs to track anything the motion model did not anticipate. That tension is the standard account of why such a coefficient has a useful floor as well as a ceiling, and it is consistent with the sweep of Section 4.4 flattening below 0.02 rather than continuing to improve. The floor was not isolated experimentally here, and the value adopted sits at the knee rather than at the smallest tested.

The representation admits multiple hypotheses. A Gaussian belief, as used by a Kalman filter, is two parameters and always a single symmetric mode; it cannot express the proposition that the robot is in one of two rooms and nowhere in between. A sample set can, by placing mass under each. This is the reason particle filters dominate indoor localisation, where repeated geometry routinely makes two locations consistent with the same laser scan.

That capability is also what makes divergence recovery worth discussing. A filter able to hold several hypotheses can back the wrong one, and standard practice is to inject random samples so an abandoned hypothesis can be recovered. Whether that is warranted is an empirical question about the environment rather than a matter of convention, and it is settled for this environment by measurement in Section 4.4: sweeping the assumed heading about the true pose produced a single sharp minimum, and no competing mode. The configuration adopted here follows from that measurement, and is described in Section 3.5.

2.5.3 Measurement Independence

The second component is the epistemic principle that a system's report about itself is not evidence for the proposition it asserts. To evaluate whether the robot arrived, the measurement must be drawn from a path that does not run through the robot's beliefs. In simulation this is available directly, as the physics engine's state for the model. The design consequence is that the evaluation harness is constructed as a separate consumer of that state rather than as a component of the robot software, and that the two paths – the robot's own estimate and the simulator's ground truth – are only ever brought together at the point of comparison. This principle determines the architecture of the validation harness described in Section 3.6.1, and it is the single methodological commitment on which the results of this thesis depend.

3 Methodology

This chapter states the research design, describes the system built as the instrument of the study, and sets out how data were collected, how they were analysed, and what ethical considerations apply.

3.1 Research Questions and Hypotheses

The research questions were stated in Section 1.4. Each is paired here with a testable hypothesis and the measurement that decides it.

Table 3.1. Research questions, hypotheses, and deciding measurements.

RQ	Hypothesis	Deciding measurement
RQ1	Intent resolution is not the limiting factor; correct place resolution exceeds 95% across varied phrasings.	Proportion of commands whose resolved place equals the intended place.
RQ2	Reported arrival and actual arrival diverge materially when measured independently.	Reported successes versus arrivals within 0.5 m of the goal by ground truth; false-positive rate between them.
RQ3	The divergence originates in localisation rather than in language, planning, or control.	Distance between the robot's believed pose and its true pose, measured per trial and conditioned on the failing trials.
RQ4	The divergence is a configuration fault and is removable.	The same protocol, re-run after correction, on the same platform and map.

3.2 Research Design

The study uses a *quantitative, quasi-experimental, within-system repeated-measures design*. A single system is exposed to a fixed suite of natural-language commands; the dependent variables are measured per trial; a treatment is then applied to the system – a set of parameter corrections derived from diagnosis – and the identical suite is repeated. Because the platform, environment, map, place registry, and command suite are held constant across the two runs, the difference between them is attributable to the treatment.

The design is quasi-experimental rather than experimental because there is no randomised control group: the same system serves as its own control, measured before and after. This is appropriate for the question being asked, which concerns a specific engineered artefact rather than a population, but it constrains the generality of the inference, and that constraint is discussed in Section 5.5.

Three independent variables are manipulated between runs: the sensor model of the particle filter, its motion-noise model, and the identity tagging of navigation outcomes. Six dependent variables are recorded per trial: the resolved place, its correctness, the outcome the

stack reported, the Euclidean distance from ground truth to the goal, the distance from the robot's believed pose to the goal, and the distance between belief and truth.

3.3 Sample

The sample is a suite of **61 natural-language commands** spanning eight destinations in the mapped environment. The suite is constructed rather than sampled from users, and is designed to exercise lexical variation rather than to estimate a population parameter. It includes:

- direct naming ("go to the kitchen");
- polite and indirect forms ("please move to the garage", "can you drive to the kitchen");
- synonyms and aliases not identical to the registry key ("head to the lounge" for `parlour`, "go park in the parking space" for `garage`, "navigate to the visitors area" for `waiting_area`);
- verb variation ("drive", "head", "move", "take the robot to").

Eight destinations were recorded in the place registry, each with a position and a heading: `parlour`, `kitchen`, `waiting_area`, `dining_room`, `master_bedroom`, `garage`, `store_area` and `home`. Every recorded place lies in a single connected traversable region of the map, with a minimum clearance to the nearest obstacle of 1.09 m against a robot inscribed radius of 0.229 m; this was verified by a connected-component analysis over the map's distance transform, so that no reported failure could be an artefact of an unreachable goal.

3.4 Tools and Instruments

3.4.1 The Robot

The platform is ROMR (Nwankwo et al., 2023), a differential-drive mobile robot. The physical robot, shown in Figure 3.1, was built by the supervising researcher with the CAD model developed by a fellow intern; the simulation model, the software stack, and the evaluation reported here are the author's contribution.

The simulated robot has a rectangular footprint of 0.521×0.608 m, modelled as a polygon rather than a circumscribing circle because a circular approximation of radius 0.42 m blocked planning through this building's doorways. Its inscribed radius is 0.229 m.

The kinematic description is written in xacro and expands to a URDF of nine links, shown in Figure 3.2. The tree is rooted at `base_link`, and every joint is fixed: this is a differential-drive base whose only articulation is the two wheels, so the description exists to place sensors relative to the chassis rather than to model a mechanism.

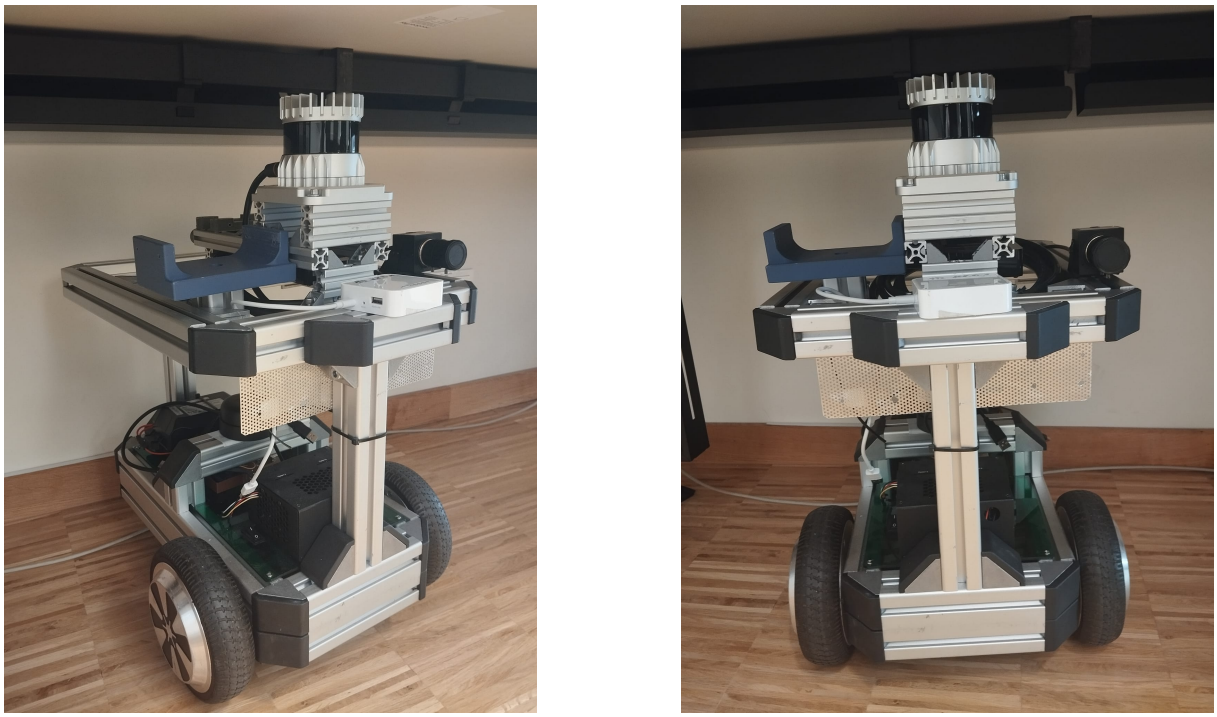

Figure 3.1. The physical ROMR platform: an aluminium-extrusion chassis carrying two driven wheels, a mast-mounted rotating lidar, a forward-facing camera, and battery and driver electronics on the lower deck.

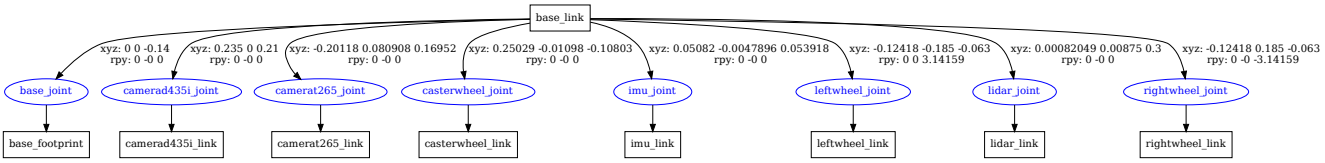

Figure 3.2. The expanded kinematic tree, generated from the model with `urdf_to_graphiz` and reproduced unaltered. Rectangles are links, ellipses are joints, and each edge carries the joint origin as translation and roll–pitch–yaw. Two values referred to elsewhere in this thesis are recorded here: the lidar sits 0.3 m above `base_link`, and the caster joint is at $z = -0.108\,03$ m, the corrected height derived from the collision mesh rather than an assumed wheel radius. `base_footprint` is the chassis projected to the ground plane and is the frame the navigation stack treats as the robot's position.

Sensing, as declared and as simulated. The two are not identical, and the difference is worth stating rather than leaving to be discovered. The model declares mounting frames for the sensor suite the physical robot carries, including an Intel RealSense D435i and a T265. The simulation implements a subset of them, listed in Table 3.2.

Two consequences follow. First, **there is no depth stream**: the D435i frame carries a plain camera sensor rather than a depth sensor, so all range information available to the system comes from the planar scan. The perception pipeline of Section 3.4.3 operates on RGB alone, and the costmap obstacle layer is configured for laser observations only. Second, the T265 frame is inert. Both were left as declared rather than removed, because the description is shared with

Table 3.2. Sensor frames declared in the kinematic description, against the sensors actually simulated.

Frame	Simulated sensor	Output	Rate
`lidar_link`	planar ray sensor	720 beams, 0.15–16 m	10 Hz
`camerad435i_link`	monocular camera	RGB 640×480	15 Hz
`imu_link`	IMU	orientation, angular rate	50 Hz
`base_link`	differential drive	wheel odometry	50 Hz
`camerat265_link`	— none —	frame only, publishes nothing	—

the physical platform; but neither contributes to any result reported in this thesis, and simulating them was judged to add failure surface and rendering cost to a study whose subject is localisation rather than perception coverage.

3.4.2 Simulation Environment and Map

Experiments run in Gazebo Classic 11. The environment is a custom indoor world of approximately 13.9×11.6 m (Figure 3.3), containing furnished rooms, doorways and corridors. The robot spawned in that world is shown in Figure 3.4.

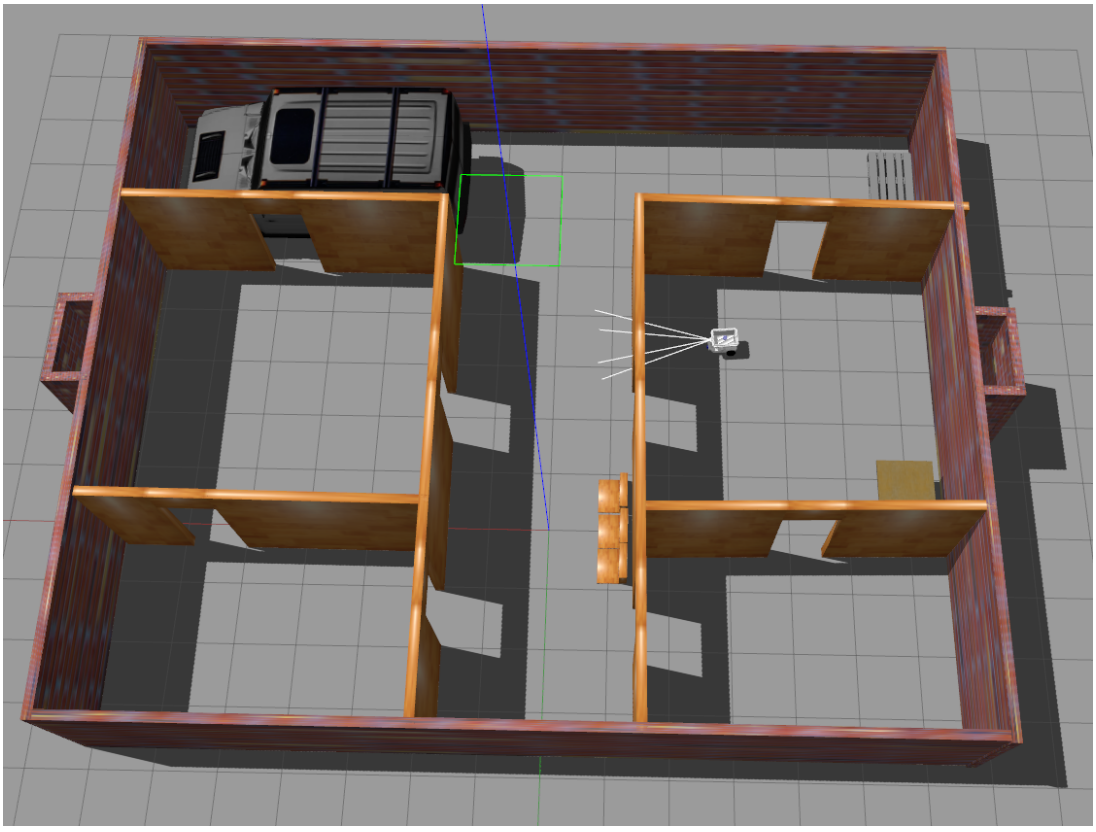

Figure 3.3. The custom Gazebo world used throughout this study: a furnished indoor environment of approximately 13.9×11.6 m *containing the eight recorded destinations.*

The occupancy grid was produced by driving the robot manually under `slam_toolbox` and is 277×232 cells at 0.05 m resolution with origin $(-6.68, -7.95)$. An earlier map was

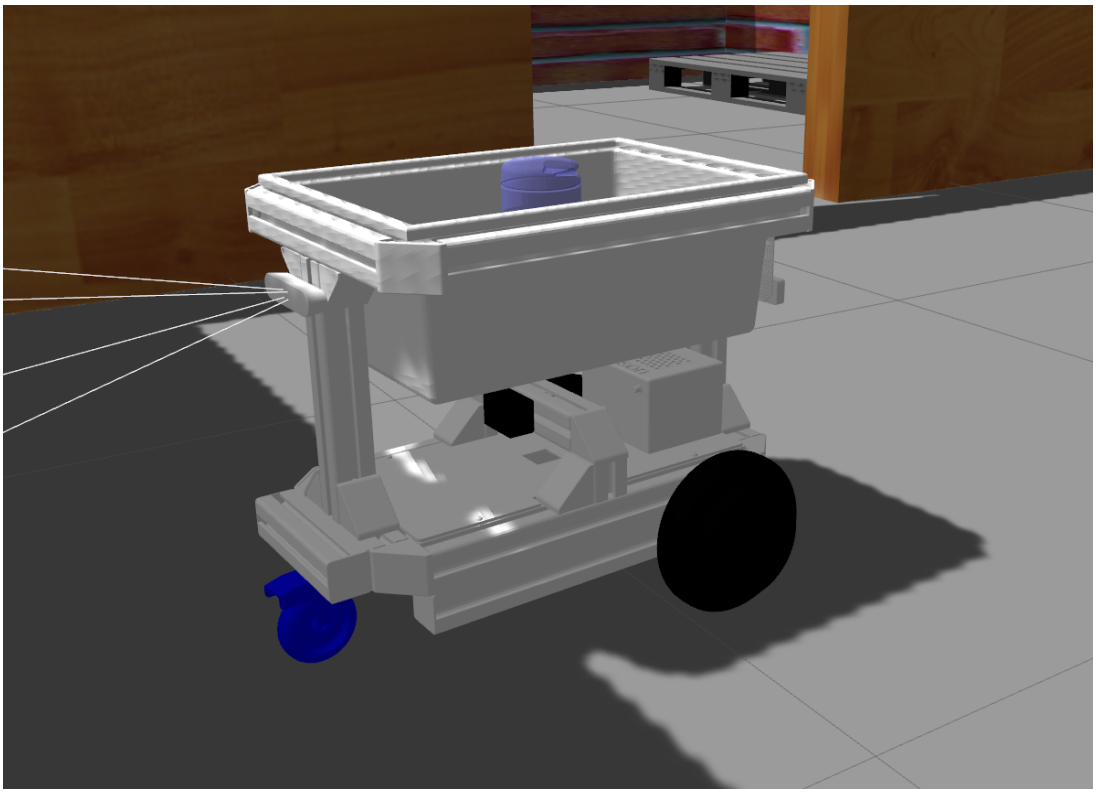

Figure 3.4. The ROMR model spawned in the simulated environment. The simulation supplies the physics, the sensor streams, and – separately from the robot software – the ground-truth pose used for evaluation.

discarded because its perimeter contained 1066 free cells on the boundary, through which the planner could route the robot outside the building; the map used here has a sealed perimeter and 6.8% unknown cells against 15.6% previously.

3.4.3 Software Stack

The system is organised in layers, shown in Figure 3.5.

Interface. A Next.js web application exposing six surfaces: navigation, chat, direct control, telemetry dashboard, vision, and a reasoning view that renders the command policy and the route a given command took through it.

Backend. A FastAPI service that is the single boundary between the browser and the robot, with seven route groups and per-client rate limiting.

Intent parsing. A hybrid design. Three phrasings are matched deterministically before any model call – requests for the current pose, the previous pose, and a camera capture – because these are the queries used to audit the robot, and a hallucinated answer to an audit question would compromise the evaluation. All other utterances are parsed by `gpt-4.1-mini` under the system prompt summarised in Listing 3.1.

The language model is used as a closed service. It is not trained, adapted or inspected here, so its internal structure is not the object of study; what matters is the shape of the contract

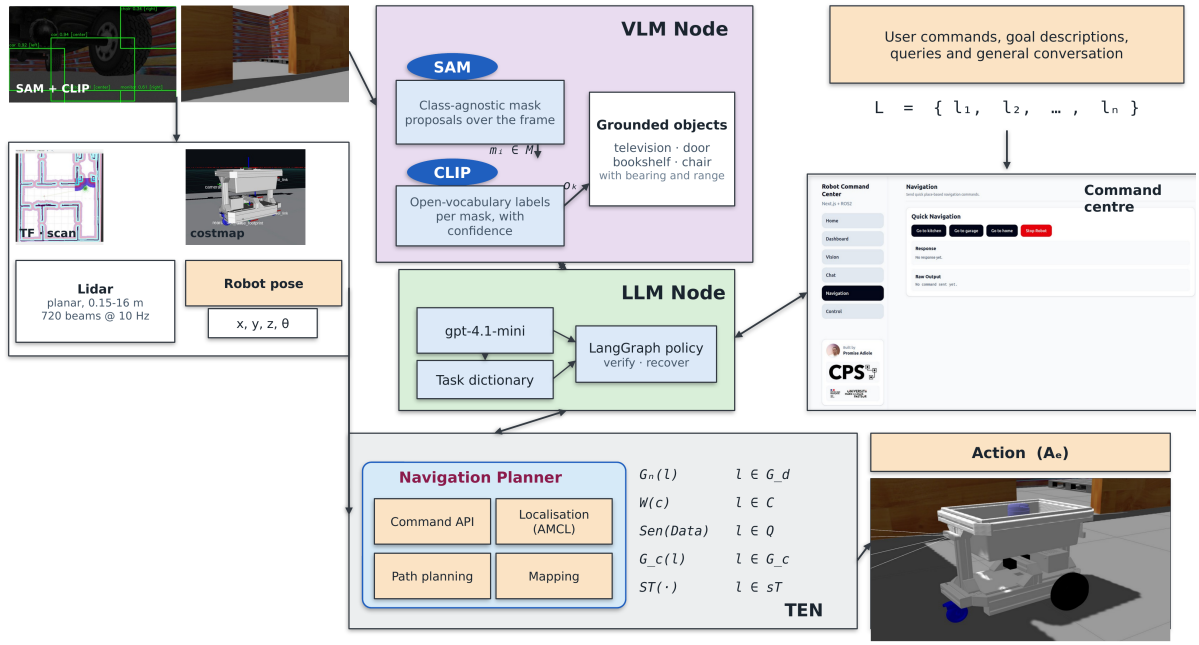

Figure 3.5. The multimodal system architecture. Sensor streams enter at the left; the VLM node proposes masks with SAM and labels them with CLIP; the LLM node parses intent and runs the command policy; the navigation planner executes; and the validation harness reads the simulator's ground-truth pose on a path independent of the robot's own estimate.

it is held to. Three constraints in that prompt do the grounding work.

```
Return ONLY valid JSON with this schema:
{
  "intent":        "...",            # one of 17 allowed values
  "target_place":  null or string,
  "route_name":    null or string,
  "motion":        null or string,
  "query":         null or string,
  "response_text": "..."
}

Allowed intents:
  NAVIGATE_TO_PLACE    FOLLOW_WAYPOINT_ROUTE  MOVE_FORWARD
  MOVE_BACKWARD        TURN_LEFT              TURN_RIGHT
  ROTATE               STOP                   GET_STATUS
  GET_POSE             GET_PREVIOUS_POSE      GET_LAST_COMMAND
  GET_SCAN_SUMMARY     LIST_VISIBLE_OBJECTS   SCENE_SUMMARY
  CAPTURE_FRAME        UNKNOWN

Rules:
  - Never invent coordinates
```

```
- Never invent place names beyond the user message
- If unclear, use UNKNOWN
```

Listing 3.1: The intent contract, abridged from `backend/app/services/intent_parser.py`.

The action space is closed. Seventeen intents are permitted and nothing else parses. The model cannot propose an action the robot has no implementation for, because there is no field in which to express one.

The model never produces a pose. *Never invent coordinates* is enforceable because the schema has no coordinate field. A target is a *name*, and that name is resolved to a metric pose by lookup in the place registry, which the model cannot write to. Grounding is therefore a database operation, not a generative one, and a hallucinated destination fails to resolve rather than sending the robot somewhere plausible-looking.

Refusal is a first-class output. `UNKNOWN` gives the model a way to decline, so an ambiguous instruction produces a request for clarification instead of a confident guess. The single command in the suite that did not resolve (Section 4.2) failed this way: it returned no destination rather than a wrong one.

The three deterministic fast-paths sit in front of this contract rather than inside it. They match requests for the current pose, the previous pose, and a camera capture — the queries used to audit the robot — so that the questions asked when checking whether the system is telling the truth cannot themselves be produced by the component under examination.

Command policy. A LangGraph state machine with six nodes, shown in Figure 3.6. Its distinguishing feature is the `verify` node, which waits for the navigation action's real outcome rather than treating goal acceptance as success, and the `answer` node, which declines to confirm an arrival when the pose covariance exceeds a threshold.

Every node records when it ran, how long it held the command, and which branch it took. That trace is built by the policy itself and returned with the reply, and is additionally exported to LangSmith for interactive inspection; because it is generated and persisted locally, no result reported here depends on that external service being reachable.

Figure 3.7 shows one command as it executed.

The proportions in that figure are the argument for the architecture. The node that costs almost nothing to add — `navigate` dispatches in 2 ms — is not where the information is; the information is in the 20.5 s that `verify` spends waiting to find out whether the dispatch achieved anything.

Perception. SAM (`vit_b`) proposes class-agnostic masks over the current camera frame; OpenCLIP (`ViT-B-32`, `laion2b_s34b_b79k`) labels each mask by embedding similarity against an open vocabulary. Captioning with a BLIP-family model was proposed and trialled during the project but is not part of the delivered system: it added a second failure surface and a substantial inference cost without contributing to goal resolution, which is driven by the place registry rather than by visual search.

Retrieval. Project documents are chunked with a token-aware splitter, embedded with

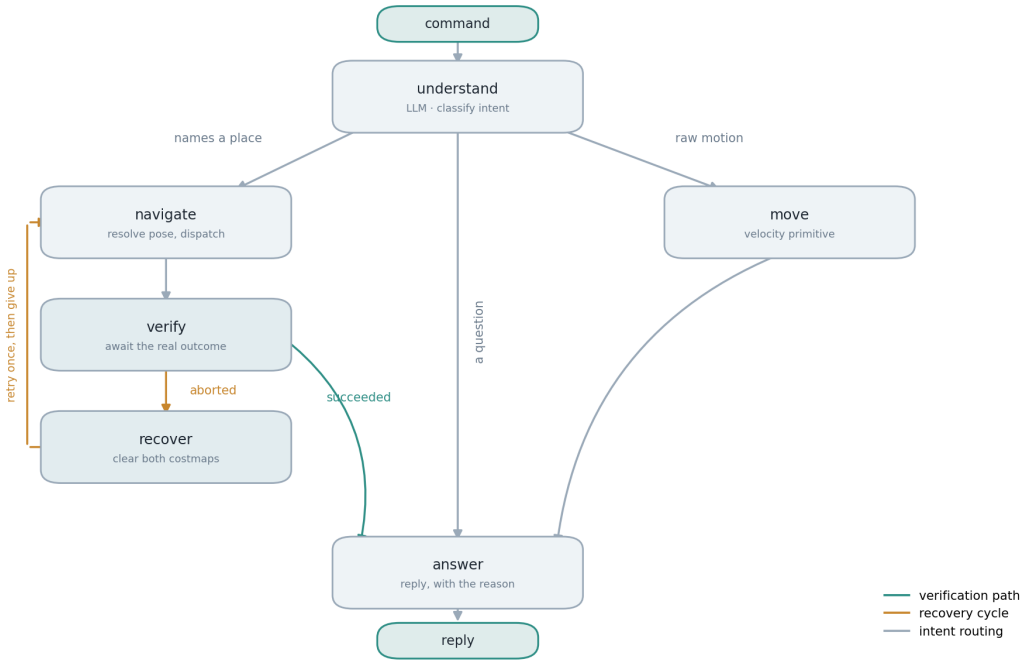

Figure 3.6. The command policy as a state machine. Intent routing sends a command down one of three branches. The verification path exists because dispatching a goal and reaching one are different events: a policy without `verify` *answers as soon as the navigation action is accepted, which is the behaviour this thesis set out to measure. The recovery cycle clears both costmaps and retries once before giving up.*

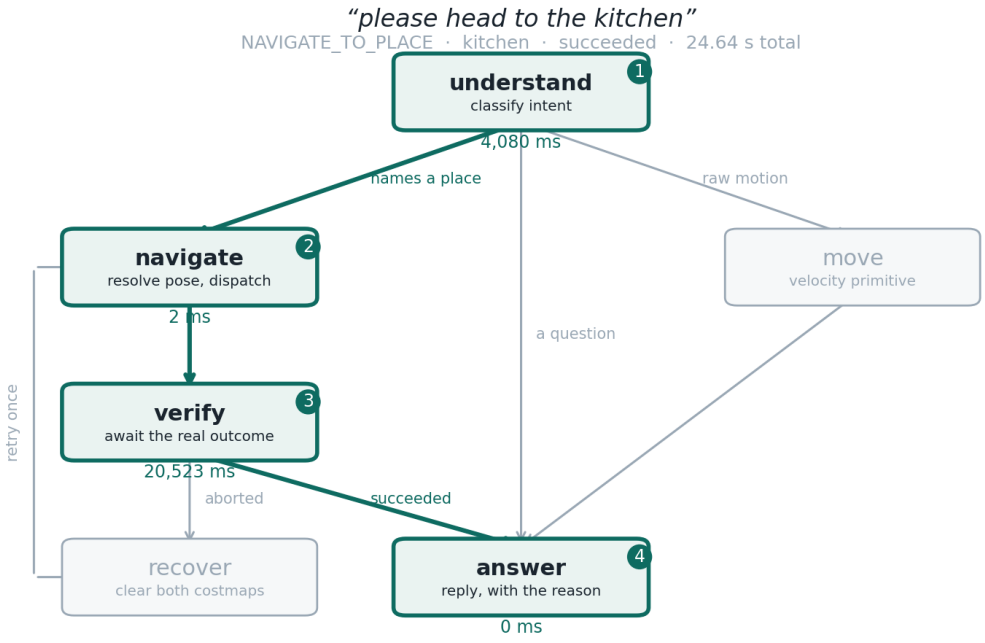

Figure 3.7. A recorded command traced through the policy. Nodes and edges the run touched are highlighted, numbered in execution order, and annotated with the time each node held the command. Almost the whole 24.64 s is spent in `verify` *waiting for the navigation outcome; intent parsing takes 4.08 s and the remaining nodes are effectively instantaneous. Drawn by* `scripts/plot_trace.py` *from the policy's own trace log.*

`text-embedding-3-large` into 3072 dimensions, and stored in a Pinecone index. Embeddings are cached by content hash so that unchanged text is never re-embedded.

Middleware. ROS 2 Humble over Cyclone DDS. The host provides no multicast route and the default domain collided with other traffic, so the stack is configured for unicast loopback on domain 42.

Navigation. Navigation 2 with AMCL for localisation, the NavFn global planner, the DWB local controller, and static, obstacle and inflation costmap layers. Inflation uses a cost scaling factor of 6.0 over a 0.55 m radius; the radius exceeds the footprint's circumscribed radius of 0.422 m, which is the distance over which collision depends on heading.

3.5 The Localisation Subsystem in Operation

Section 2.5.2 set out the particle representation. This section shows it as it runs on this robot, because the quantities the evaluation depends on — the believed pose and its covariance — are computed from the sample set and are not meaningful without it.

Figure 3.8 shows the whole subsystem at once, as the operator sees it. Every element this section goes on to describe separately is visible there simultaneously, which is worth establishing before any of them is isolated.

Three things in that figure are worth drawing out, because each is a decision recorded elsewhere in this chapter.

The footprint is a polygon. The robot is modelled as the rectangle 0.521×0.608 m rather than by a circumscribing circle of radius 0.422 m. The difference matters at doorways: a circular approximation of that radius blocked planning through openings this robot passes through, because it reserves clearance the robot does not need at the orientation it actually travels in.

The inflated band is not an obstacle. Only the inner cyan region is treated as blocked. The purple gradient is expensive rather than impassable, and its decay rate is set by the cost scaling factor. Measured against this map, 25.3% of the costmap is at or above the value the planner refuses, while all eight recorded destinations return a cost of zero.

The particle cloud has physical extent. It is drawn at the same scale as the rooms, so its size can be read against the building. A cloud spanning a doorway means the filter cannot distinguish which side of that doorway the robot is on — and every layer above it will nonetheless act on the single mean pose derived from it.

3.5.1 Dimensionality

The textbook presentation of a particle set is one-dimensional: the state is a position on a line and the belief is a row of spikes along it. This robot drives on a floor, so each hypothesis is a planar pose,

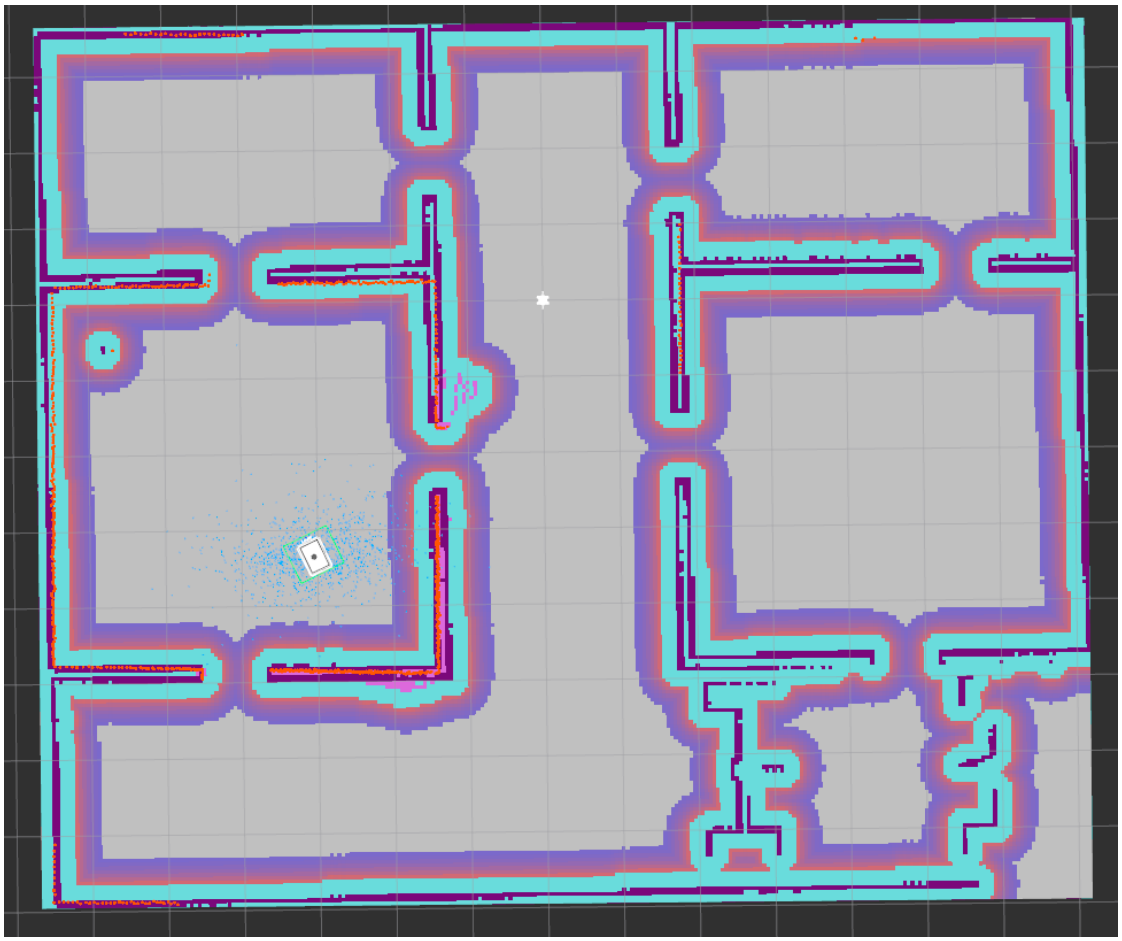

Figure 3.8. The navigation subsystem in operation, with the robot spawned and AMCL active. Grey is free space in the static map. The cyan band hugging each wall is the region within the robot's inscribed radius, which the planner treats as blocked; the purple gradient outside it is the inflation layer, whose cost decays with distance and steers paths toward the middle of a corridor rather than along its edge. Orange points are live laser returns, lying on the mapped walls — the agreement quantified in Section 4.4. At the left of the plan the robot's footprint is drawn as the rectangular polygon it is modelled by, rather than a circumscribing circle, surrounded by the blue particle cloud: the same sample set plotted analytically in Figures 3.10–3.12, here shown in place. The extent of that cloud relative to the rooms around it is the robot's uncertainty about which part of the building it occupies.

$$x^{[j]} = \left(x^{[j]},\, y^{[j]},\, \theta^{[j]}\right) \in SE(2), \tag{3.1}$$

and the sample set is therefore a cloud in three dimensions rather than a row of ticks. The reported uncertainty follows from the weighted moments of that cloud,

$$\mu = \sum_j w^{[j]} x^{[j]}, \qquad \Sigma = \sum_j w^{[j]} \left(x^{[j]} - \mu\right)\left(x^{[j]} - \mu\right)^{\top}, \tag{3.2}$$

from which $\sigma_x = \sqrt{\Sigma_{11}}$, $\sigma_y = \sqrt{\Sigma_{22}}$ and $\sigma_\theta = \sqrt{\Sigma_{66}}$. Every covariance figure reported in Chapter 4 is one of these three numbers.

Headings require care that positions do not. Because θ lives on a circle, its arithmetic

mean is not its mean direction — averaging 359° and 1° yields 180°, which is opposite to both. The mean heading is therefore computed as the direction of the summed unit vectors, and the spread as the root weighted mean square of the wrapped deviations.

3.5.2 Configuration

The parameters governing the sample set are given in Listing 3.2. Their values are derived in Section 4.4; they are reproduced here so that the figures which follow can be read against the settings that produced them.

```
min_particles: 500      # J, lower bound
max_particles: 2000     # J, upper bound; adaptive between the two
update_min_d: 0.25      # metres travelled before an update
update_min_a: 0.2       # radians rotated before an update
resample_interval: 1    # resample on every update

alpha1: 0.02            # rotation noise from rotation
alpha2: 0.02            # rotation noise from translation
alpha3: 0.05            # translation noise from translation
alpha4: 0.05            # translation noise from rotation

max_beams: 120          # beams scored per update, of 720
z_hit:  0.8             # weight on "hit the predicted wall"
z_rand: 0.2             # weight on "this beam is noise"
```

Listing 3.2: Sample-set and motion-noise parameters, from `config/nav2_params.yaml`.

These parameters divide along the line drawn in Section 2.5.2. The differential-drive motion model is the *proposal*: each particle is pushed through the odometry with noise, and α_1–α_5 set how widely that push scatters them. Because the proposal is the motion model, the prior and transition terms cancel in Equation (2.6) and the weight reduces to the measurement likelihood, whose parameters are `z_hit`, `z_rand`, `sigma_hit`, `max_beams` and the range limits. The alphas are therefore a claim about how far the odometry may be trusted; the sensor parameters are claims about the scanner, and can be checked against it.

The particle count is adaptive, which is the sense in which the method is *adaptive* Monte Carlo localisation: the filter draws more samples when the posterior is diffuse and fewer when it is concentrated. Both figures in this section were captured with J at its ceiling of 2000.

3.5.3 The Sample Set as Function Approximation

Figure 3.9 shows the belief in the form the theory describes: a rug of individual particles beneath a density reconstructed from them. Nothing in the filter holds the curve. It is drawn here by kernel density estimation over the samples, precisely to make the point that the samples are the belief and the curve is inference about them.

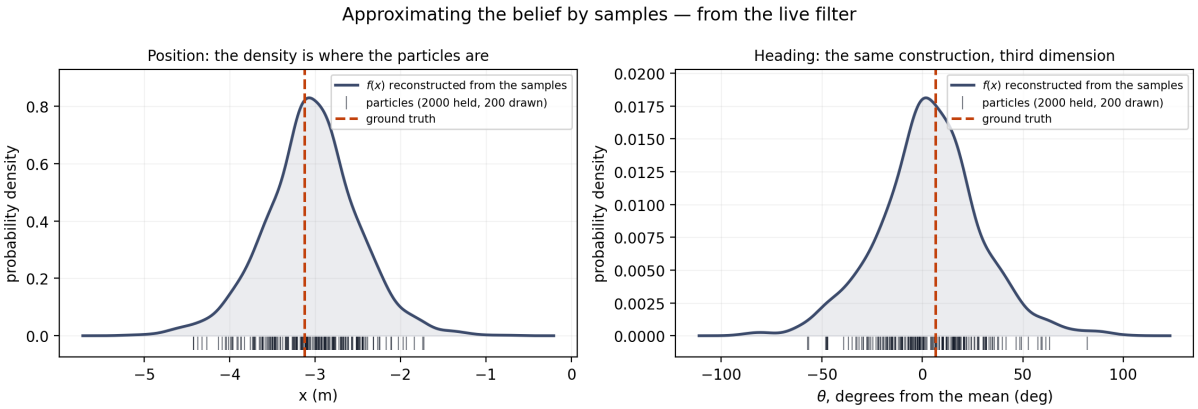

Figure 3.9. The belief as an approximation by samples, captured from the running filter. Each tick is one particle; the curve is reconstructed from the ticks rather than stored. Left, the marginal over x; right, the marginal over heading. Because the filter resamples on every update, all 2000 particles carried identical weight, so the probability of a region is expressed by how many samples fall in it.

3.5.4 The Sample Set Under Motion

Figures 3.10 and 3.11 show the same cloud immediately after the filter is seeded from ground truth, and again after a 90° rotation — the manoeuvre identified in Section 4.4 as the one that degrades the estimate.

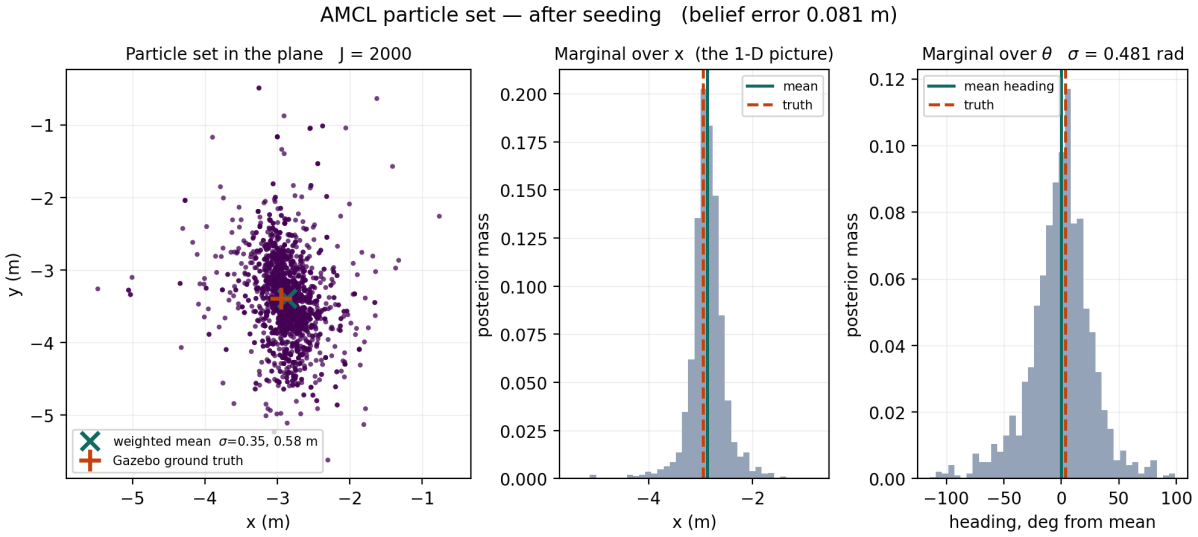

Figure 3.10. The particle set immediately after seeding from ground truth. Left, all 2000 hypotheses in the plane with the weighted mean and the simulator's true pose; centre, the marginal over x; right, the marginal over heading. Belief error 0.081 m.

Two features of the pair are worth noting. The heading spread does not grow: 0.481 rad before the turn and 0.432 rad after it, against the ninefold growth recorded in Table 4.5 under the original coefficients. The positional spread instead *reorients* — 0.349 m by 0.583 m becomes 0.527 m by 0.297 m — because uncertainty along a corridor differs from uncertainty across it, and rotating the robot rotates that ellipse. Neither figure is evidence that the filter is correct; both

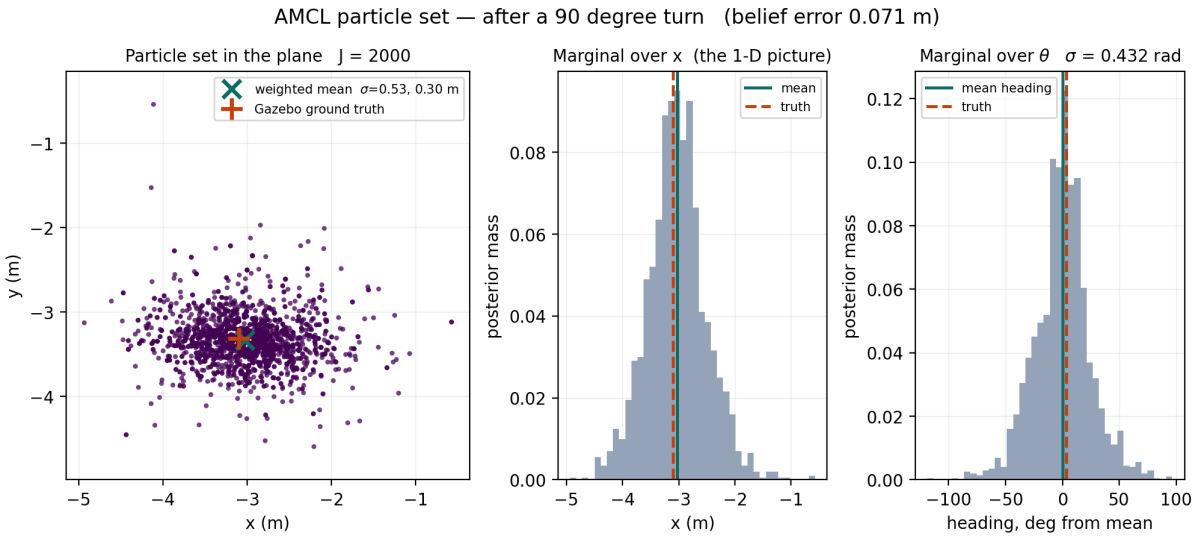

Figure 3.11. The same set after a 90° rotation, with the corrected motion-noise coefficients of Listing 3.2. Heading spread is 0.432 rad against 0.481 rad before the turn, and belief error is 0.071 m. Under the coefficients used before correction the same manoeuvre multiplied heading spread by 9.3 (Table 4.5).

are evidence of what it believed, which is the quantity Chapter 4 tests against ground truth.

3.5.5 The Same Manoeuvre Under the Original Coefficients

Figures 3.10 and 3.11 show the filter as configured after correction, and on their own they establish only that it behaves acceptably. To show that the coefficients are responsible, the experiment was repeated with $\alpha_1 = \alpha_2$ returned to 0.25 — the value trialled during diagnosis — on the same robot, the same map and the same 90° rotation, re-seeding from ground truth beforehand so that both conditions start from the same place. The result is Figure 3.12.

Table 3.3. The same 90° rotation under the two settings, seeded from ground truth in both cases.

	$\alpha_1 = 0.02$	$\alpha_1 = 0.25$
σ_θ before the turn (rad)	0.481	1.358
σ_θ after the turn (rad)	**0.432**	**1.317**
$\sigma_x \times \sigma_y$ after (m)	0.53×0.30	0.75×0.88
Belief error after (m)	**0.071**	**0.321**

The comparison isolates the coefficient. Under the original setting the heading spread was already 1.358 rad before the deliberate rotation, because the brief turn used to make the filter converge after seeding was by itself enough to disperse it. Positional spread is roughly two and a half times larger in area, and belief error four and a half times larger.

The visual difference in the heading marginal is the substantive one. At $\alpha_1 = 0.02$ the mass is concentrated within roughly 50° of the mean; at 0.25 it is distributed across the full circle, which is to say the filter is close to having no opinion about orientation at all. A robot in that

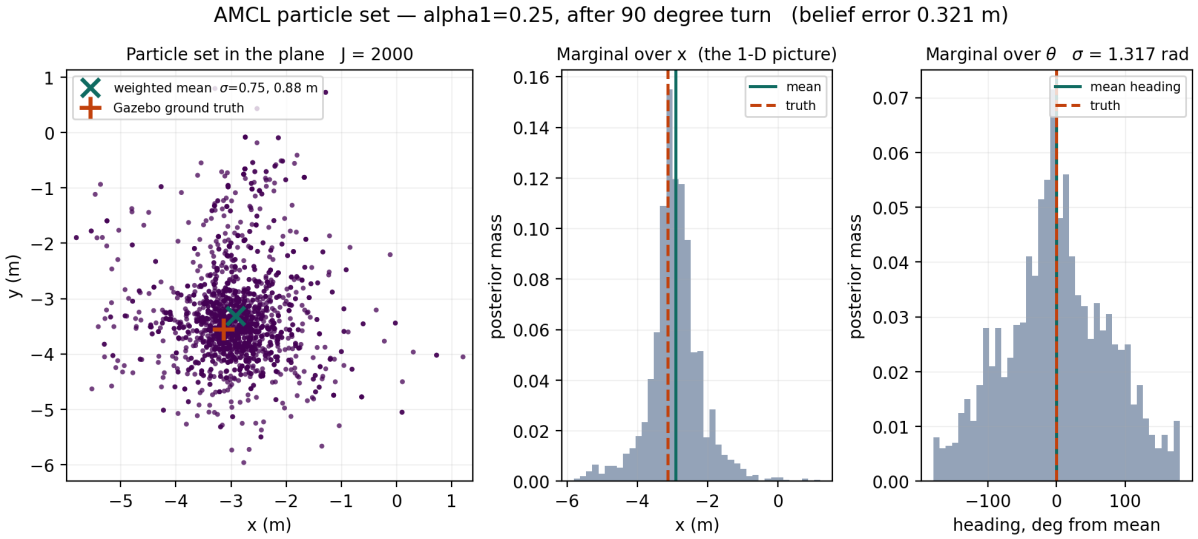

Figure 3.12. The particle set after the same 90° rotation with $\alpha_1 = \alpha_2 = 0.25$. The cloud spans several metres and the heading marginal carries mass across the entire circle: the robot has no useful estimate of which way it is facing. Compare Figure 3.11, the identical manoeuvre at $\alpha_1 = 0.02$.

state can still report a pose, and every layer above it will use that pose. This is the mechanism behind the false arrivals reported in Section 4.3, shown in the representation rather than inferred from the outcome.

3.5.6 An Operational Property Worth Recording

AMCL publishes the sample set only when the filter updates, and it updates only once the robot has passed `update_min_d` or `update_min_a`. A stationary robot therefore publishes nothing at all, on a topic that has an active publisher and a correctly configured subscriber. This was encountered twice in this work: once when a monitoring component judged a parked robot to be poorly localised because its estimate was old, and once when the capture tool that produced these figures returned nothing until it was made to rotate the robot while waiting. Both are recorded here because absence of traffic on such a topic reads as a fault and is not one.

3.6 Data Collection

3.6.1 The Validation Harness

Arrival is measured by a harness that is a separate consumer of the simulator's state, not a component of the robot software. For each trial it:

1. issues the phrasing to the backend over HTTP, exactly as a user would;
2. records the place the language layer resolved, and whether that matches the intended destination;

3. records the outcome the navigation stack reported;

4. reads the robot's true pose from the simulator by direct query;

5. reads the robot's believed pose from the localisation topic;

6. computes three distances.

Writing $\mathbf{p}_{\text{true}}$ for the simulator's pose, $\mathbf{p}_{\text{belief}}$ for the filter's estimate and $\mathbf{g}$ for the goal, all in the map frame:

$$e_{\text{true}} = \|\mathbf{p}_{\text{true}} - \mathbf{g}\|_2 \tag{3.3}$$

$$e_{\text{amcl}} = \|\mathbf{p}_{\text{belief}} - \mathbf{g}\|_2 \tag{3.4}$$

$$d_{\text{drift}} = \|\mathbf{p}_{\text{true}} - \mathbf{p}_{\text{belief}}\|_2 \tag{3.5}$$

A trial counts as an *arrival* when $e_{\text{true}} \leq 0.5\,\text{m}$, and as a *false positive* when the stack reported success and $e_{\text{true}} > 0.5\,\text{m}$. The threshold is a choice; its sensitivity is reported in Section 4.3.1.

The independence of the measurement rests on Equation (3.3) being computed from a quantity the robot never touches. The ground-truth pose is obtained from the physics engine's model state, which shares no sensor, no filter and no coordinate frame with the particle filter that produces $\mathbf{p}_{\text{belief}}$.

3.6.2 Trial Protocol

Each trial begins from a defined localisation state. If the robot has left the mapped area it is returned to the spawn pose; if belief and truth disagree by more than 0.35 m the estimate is re-seeded from ground truth. Both events are counted and reported, because needing them is itself a result. Commands are issued through the policy endpoint, which waits for the real navigation outcome with a budget of 90 s and permits one recovery attempt.

3.6.3 Supporting Measurements

Three additional measurements support the diagnosis in Chapter 4.

Scan–map agreement. For a scan taken at the true pose, each beam endpoint is transformed into the map frame and its distance to the nearest occupied cell computed via a chamfer distance transform. The distribution of those distances characterises how informative the scan is, independently of the filter.

Localisation step response. The filter is seeded from ground truth and then subjected to isolated motions – standing still, rotating in place, translating – with position error, heading error and covariance recorded after each, isolating which motion degrades the estimate.

Parameter sweep. The rotational noise coefficient is swept over five values, re-seeding from ground truth before each, with error and covariance after a fixed 90° rotation recorded for each value.

3.7 Data Analysis

Per-trial records are written to CSV and analysed with descriptive statistics. Because the error distributions are heavily right-skewed – a mislocalised robot can be metres from its goal while a successful one is within centimetres – the median is reported as the primary measure of central tendency, with the mean and maximum given alongside so that the skew is visible.

Rates are reported as proportions with their integer numerators and denominators, so that the sample size behind each percentage is legible. The false-positive rate is computed over reported successes only, that is $|\{\text{reported} \wedge e_{\text{true}} > 0.5\}| \,/\, |\{\text{reported}\}|$, and is reported separately from the arrival rate over all trials, since conflating the two was found to produce a misleading figure.

3.7.1 Arrival as a Decision

Reporting the false-positive rate alone describes one direction of error and leaves the other unnamed. Since the corrected system was found to fail in the opposite direction, both are stated here in the standard form.

For a trial t, let $A(t)$ be true when the robot physically arrived, and $R(t)$ be true when the navigation stack reported success:

$$A(t) \equiv \left[\, e_{\text{true}}(t) \le 0.5\,\text{m} \,\right], \qquad R(t) \equiv \left[\, \text{outcome}(t) = \texttt{succeeded} \,\right] \tag{3.6}$$

A is decided by the simulator, R by the robot. The two are independent by construction, which is what makes their disagreement measurable. Counting trials in the four cells,

$$\begin{aligned} \text{TP} &= |\{t : R(t) \wedge A(t)\}|, & \text{FP} &= |\{t : R(t) \wedge \neg A(t)\}|, \\ \text{FN} &= |\{t : \neg R(t) \wedge A(t)\}|, & \text{TN} &= |\{t : \neg R(t) \wedge \neg A(t)\}|, \end{aligned} \tag{3.7}$$

from which

$$\text{precision} = \frac{\text{TP}}{\text{TP} + \text{FP}}, \qquad \text{recall} = \frac{\text{TP}}{\text{TP} + \text{FN}}. \tag{3.8}$$

Precision is the quantity of interest for the failure this thesis reports: it is the probability that a claimed arrival is real, and its complement is the false-positive rate quoted throughout. Recall is the probability that a real arrival is acknowledged. The distinction matters operationally, because the two failures are not equally serious. A false positive cannot be detected from inside the system — nothing downstream has evidence to contradict it. A false negative is visible in the robot's own report and costs time rather than correctness.

Trial duration is used as a validity check. A navigation in this environment takes tens of seconds; a trial returning in a few seconds indicates the outcome was not produced by the navigation it purports to describe. This check is applied to all runs and its consequences are reported in Section 4.1.

Formal inferential statistics are not applied. The comparison is between two runs of a deterministic-by-design system rather than between samples from two populations, and the effect sizes involved – a median error moving from metres to centimetres – do not require a significance test to be interpretable. This choice is revisited in Section 5.5.

3.8 Ethical Issues

Human participants. No human participants were involved. The command suite is author-constructed, and no user study was conducted. This removes consent and data-protection obligations, but it also removes any claim that the phrasings are representative of how non-expert users would actually speak to a robot; that limitation is stated in Section 5.5 rather than obscured.

Physical safety. All experiments were conducted in simulation. No person or property was exposed to a moving robot. Had the hardware deployment succeeded, the safety case would have required the arrival-verification mechanism described here, which is part of the motivation for building it.

Honesty in reporting. The commitment here is to report the measurement obtained rather than the one desired. Two instances are recorded. The initial evaluation produced a strongly negative headline result, reported as the principal finding rather than quietly repaired. A later audit of that same evaluation found most of its trials had not executed a navigation at all, reducing the effective sample from 54 to 16; that reduction is reported in Section 4.1 and the affected figures restated, although it weakens the headline claim's denominator.

Attribution. The physical robot was built by the supervising researcher and its CAD model developed by another intern. The contribution claimed in this thesis is the simulation model, the software stack, the evaluation harness, and the analysis.

Data and reproducibility. Every figure and table in Chapter 4 derives from per-trial CSV records shipped with the source, along with the scripts that produce them. No data were excluded from a reported aggregate except under an explicit stated criterion, and in those cases both filtered and unfiltered values are given.

Model use and cost. The system calls commercial language and embedding APIs. Content-addressed embedding caches and a per-client rate limit bound both the cost and the request volume.

4 Results

This chapter reports the findings. Interpretation is deferred to Chapter 5; what follows is what was measured.

All results in this chapter are obtained in Gazebo simulation, on the platform and in the environment described in Chapter 3. No result reported here was obtained on physical hardware; the attempted hardware deployment and the reason it did not proceed are given in Section 6.5.1.

4.1 Validity Audit of the Initial Run

The initial evaluation recorded 54 trials. Applying the duration check defined in Section 3.7 to those records gives a median trial duration of 5.1 s, with 38 of 54 trials returning in under 10 s. A navigation in this environment, measured after the corrections reported in Section 4.5, has a median duration of 57.2 s.

Partitioning the initial run on that criterion gives Table 4.1.

Table 4.1. The initial run partitioned by trial duration. All reported successes fall in the group whose durations are consistent with an executed navigation.

	$\geq$ **10 s**	$<$ **10 s**
Trials	16	38
Reported success	12	0
Arrived ($e_{\text{true}} \leq 0.5$ m)	3	0
Median e_{true} (m)	6.35	5.98
Median d_{drift} (m)	6.25	0.54

All 12 reported successes fall in the 16-trial group. The 38 short-duration trials reported no successes and therefore contribute nothing to the false-positive ratio. Aggregate figures for the initial run are consequently reported over the 16-trial group throughout this chapter.

4.2 RQ1: Language Understanding

Across the 61-command suite, 60 commands resolved to the intended destination, a rate of 98%. The single failure was a phrasing that resolved to no registry entry rather than to a wrong one.

Resolution was correct for phrasings that did not contain the registry key, including *lounge*, *living room* and *sitting room* for `parlour`; *car park* and *parking space* for `garage`; *visitors area* for `waiting_area`; and *storage* for `store_area`. The same rate was obtained in the initial run, where all trials resolved correctly.

4.3 RQ2: Reported Versus Actual Arrival

Table 4.2 reports the two quantities separately for the initial run.

Table 4.2. Reported and actual arrival in the initial run, over the 16 trials whose duration is consistent with an executed navigation.

Quantity	Count	Rate
Trials	16	—
Reported success by the stack	12	75%
Arrived within 0.5 m	2 of those 12	17%
Reported success that was false	10 of 12	**83**%

Ten of the twelve reported successes placed the robot more than 0.5 m from the goal. Median true error over the 16 trials was 6.35 m; the maximum was 15.66 m.

Figure 4.1 places that result beside the corrected system of Section 4.5, on the same axes.

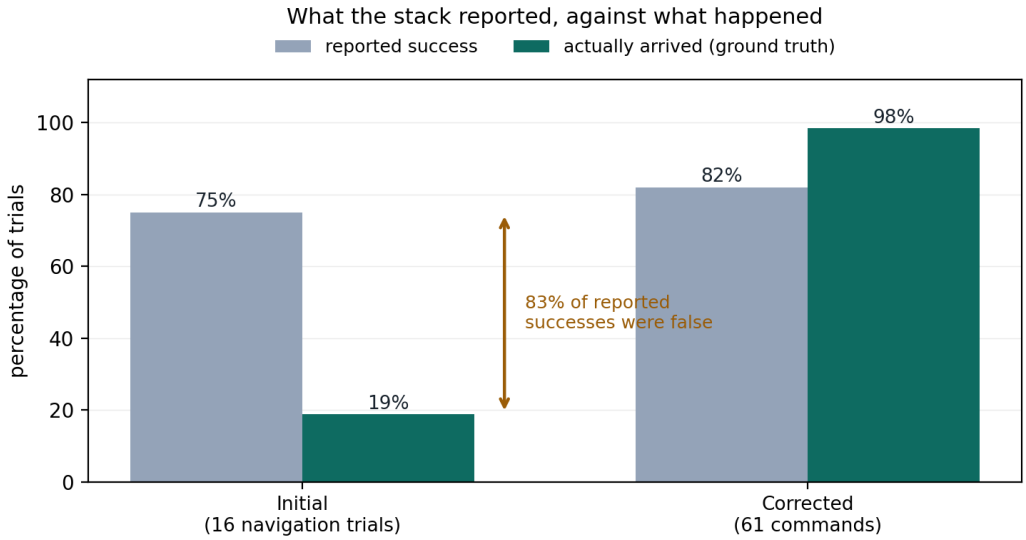

Figure 4.1. Reported success against actual arrival, for the initial and corrected systems. Reported success is what the navigation stack declared; actual arrival is scored at 0.5 m against the simulator's pose for the robot. In the initial system the two diverge by a factor of four over the same trials; in the corrected system the ordering reverses, and the stack reports fewer successes than occur.

4.3.1 Sensitivity to the Arrival Threshold

Table 4.3 reports arrival counts over the full initial record at four thresholds.

Figure 4.2 shows where the trials actually finished in each system, rather than only whether they crossed the threshold.

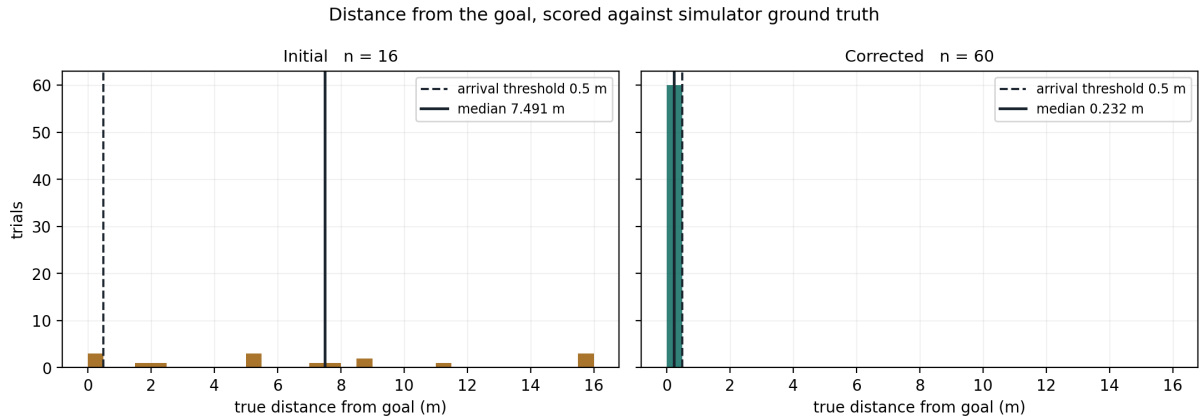

Figure 4.2. Distribution of true distance from the goal. Left, the initial system: mass spread across the building, with a median of 6.35 m and a tail past 15 m. Right, the corrected system: every trial inside 0.5 m. Both panels share an axis, so the difference is one of kind rather than degree.

Table 4.3. Arrivals as a function of the threshold, initial run.

Threshold (m)	0.3	0.5	0.6	1.0
Arrivals	0	3	3	4

4.4 RQ3: Attribution of the Discrepancy

4.4.1 Goal Reachability

A connected-component analysis over the map's distance transform, thresholded at the robot's inscribed radius of 0.229 m, places all eight recorded destinations and the spawn pose in a single traversable region. The minimum clearance among the eight is 1.09 m. Probing the live global costmap returned a cost of zero at all eight destinations, against 25.3% of the costmap at or above the value the planner treats as blocked.

4.4.2 Scan–Map Agreement

For a scan of 720 beams taken at the true pose, Table 4.4 reports the distance from each beam endpoint to the nearest occupied map cell.

Table 4.4. Scan–map agreement at the true pose, 720 beams.

Mean distance to nearest wall	0.047 m
Median distance	0.050 m
Within 0.10 m	66.8%
Within 0.20 m	100%

Rotating the assumed heading about the true pose and recomputing gives a mean of 0.047 m at zero offset against 0.443 m at 45°, 0.410 m at 90°, 0.762 m at 135° and 0.749 m at

180°. The minimum is unique and occurs at the correct heading.

4.4.3 Localisation Step Response

Seeding the filter from ground truth and applying isolated motions gives Table 4.5.

Table 4.5. Position error, heading error and heading standard deviation after isolated motions, seeded from ground truth.

Condition	e (m)	$\Delta\theta$ (°)	σ_θ (rad)
Immediately after seeding	0.014	−0.1	0.128
After 8 s at rest	0.015	−0.1	0.128
After one 90° turn	0.159	−17.4	**1.189**
After turning back	0.577	−24.9	1.233
After 0.8 m forward	0.736	−36.5	1.105
After 0.8 m reverse	0.289	−26.8	0.883

Standing still for 8 s left the estimate unchanged. A single 90° rotation multiplied the heading standard deviation by a factor of 9.3.

4.4.4 Rotational Noise Parameter Sweep

Sweeping the rotational noise coefficient $\alpha_1 = \alpha_2$, re-seeding from ground truth before each trial and applying a fixed 90° rotation, gives Table 4.6.

Table 4.6. Position error, heading error and heading standard deviation after a 90° rotation, as a function of the rotational noise coefficient.

$\alpha_1 = \alpha_2$	e (m)	$\Delta\theta$ (°)	σ_θ (rad)
0.25	0.469	−13.3	1.257
0.10	0.279	−9.5	0.958
0.05	0.168	+1.0	0.694
0.02	**0.108**	−2.8	**0.441**
0.01	0.104	−1.8	0.364

The relationship is monotonic and decreasing over the swept range, flattening below 0.02.

Figure 4.3 plots both columns. The direction is the substantive result: the value trialled first, 0.25, was chosen because the robot's fixed casters scrub during rotation and the filter should therefore trust rotational odometry less. It is the worst point on the curve.

4.4.5 Configuration Faults Identified

Four faults were identified and are listed in Table 4.7 with the measurement that exposed each.

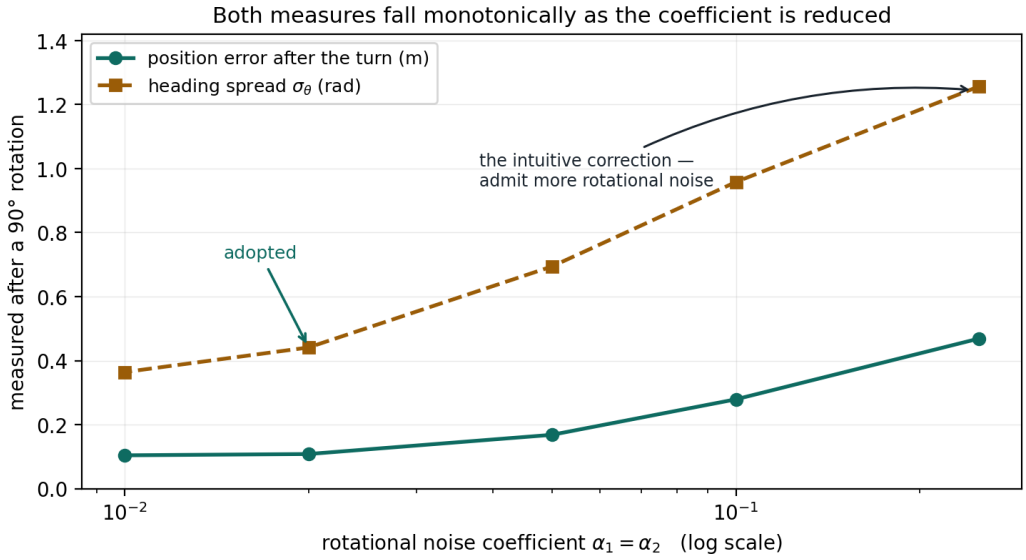

Figure 4.3. Position error and heading spread after a 90° rotation, against the rotational noise coefficient, seeded from ground truth before each trial. Both fall monotonically as the coefficient is reduced. The coefficient adopted is 0.02 *rather than* 0.01*: the motion noise is the only source of particle diversity between resamplings (Section 2.5.2), so the coefficient has a floor as well as a ceiling, and* 0.02 *sits at the knee.*

Table 4.7. Configuration faults and the measurement that exposed each. Half *records which side of Equation (2.6) the fault sat on: a proposal fault degrades the estimator while still targeting the right posterior, whereas a weight fault changes what is being estimated.*

Fault	Half	Was → Is	Exposed by
Sensor model range cutoff	weight	8 m → 16 m	Fitted scanner reaches 16 m in a 13.9 m building
Rotational noise coefficient	proposal	0.20, then 0.25 → 0.02	Sweep, Table 4.6
Hit-versus-noise weighting	weight	0.5/0.5 → 0.8/0.2	Scan–map agreement, Table 4.4
Navigation outcome identity	—	untagged → per-goal epoch	Duration audit, Section 4.1

An intermediate run, conducted after the three localisation faults were corrected but before the outcome-identity fault was, recorded a mean belief-to-truth drift of 0.146 m against 2.563 m previously, with an arrival rate of 11% and a median trial duration of 5.4 s.

4.5 RQ4: The Protocol Re-Run After Correction

Table 4.8 reports the identical 61-command protocol before and after correction, on the same platform, map and place registry.

Applying the duration check to the corrected run leaves 54 trials of at least 10 s; within that subset, 54 of 54 arrived and 0 of 44 reported successes were false. The seven short-duration

Table 4.8. The same evaluation protocol and definitions applied to the initial and corrected systems. Arrival threshold 0.5 m, scored against simulator ground truth.

Measure	**Initial**	**Corrected**
Trials analysed	16	61
Intent resolved correctly	100%	60/61 (98%)
Reported success	12 (75%)	50 (82%)
Arrived within 0.5 m	2	**60** (98%)
False positives among reported	10/12 (83%)	**0/50**
Median e_{true} (m)	6.35	**0.231**
Mean e_{true} (m)	—	0.259
Maximum e_{true} (m)	15.66	0.483
Median d_{drift} (m)	6.25	0.108
Mean d_{drift} (m)	6.94	0.189
Median trial duration (s)	5.1	57.2
Re-localisations required	—	0/61
Robot found outside the map	—	0/61

trials in the corrected run all arrived, with true errors between 0.19 m and 0.48 m.

4.5.1 Precision and Recall

Applying Equations (3.6)–(3.8) to both runs gives Table 4.9.

Table 4.9. The navigation stack's success report treated as a decision about arrival, scored against ground truth. Counts are trials.

	Initial (n = 16)	**Corrected** (n = 61)
True positives	2	50
False positives	10	**0**
False negatives	1	**10**
True negatives	3	1
Precision	0.167	**1.000**
Recall	0.667	0.833

Precision rose from 0.167 to 1.000; recall rose from 0.667 to 0.833. In the corrected system every false positive has been eliminated, and the remaining error is entirely of the opposite kind: ten arrivals that occurred and were not reported.

4.5.2 Residual Discrepancy

In the corrected run the stack reported 50 successes while 60 arrivals occurred. Ten trials therefore arrived without the navigation stack confirming it. Table 4.10 reports those trials.

Instrumenting one such case at `waiting_area` recorded the robot between 0.09 m and 0.24 m of a 0.25 m position tolerance for 46 s, with commanded linear velocity flat at 0.000 m s^{-1}

Table 4.10. Trials in the corrected run that arrived by ground truth but were not reported as successes.

Destination	Count	e_{true} range (m)	Duration (s)
`waiting_area`	4	0.12–0.23	≈ 185
`home`	3	0.22–0.30	≈ 185
`store_area`	2	0.25–0.39	140–185
`garage`	1	0.25	167

and commanded angular velocity consistently of one sign.

4.6 Perception

On a representative frame at 640 × 480, the perception pipeline returned four labelled objects: *television* at confidence 0.70 and 0.63, *door* at 0.48, and *bookshelf* at 0.48, each with a bearing class. The same detections were rendered as an annotated overlay and phrased as a scene description for the conversational interface.

4.7 Traceability

Every command executed through the policy endpoint produced a recorded route through the state machine. A representative navigation command produced the route `understand` → `navigate` → `verify` → `answer`, with a total elapsed time of 84.18 s, of which 82.11 s was spent in `verify` awaiting the navigation outcome.

In that trial the `answer` node declined to confirm arrival, reporting that the position standard deviation was 0.59 m against a limit of 0.60 m and the heading standard deviation 0.38 rad against a limit of 0.35 rad. Ground truth placed the robot 0.25 m from the goal.

5 Discussion

This chapter interprets the findings of Chapter 4, compares them with previous work, accounts for where they agree and differ, and argues their importance.

5.1 What the Results Mean

5.1.1 Language Understanding Was Never the Bottleneck

Intent resolution was correct for 60 of 61 commands, including phrasings that shared no lexical item with the registry key. Taken together with the initial run, in which every command also resolved correctly while only two arrivals occurred, this establishes the central asymmetry of the system: the language layer performed at ceiling while the system as a whole performed at 6%.

The consequence is that end-to-end success rate is an actively misleading metric for a composed system of this kind. A reader given only the initial end-to-end figure would reasonably conclude that language grounding had failed, and would allocate effort to prompt design, intent schemas, or a larger model – none of which would have moved the number, because the resolved place was already right. This is the concrete argument for attributing failure to a layer before attempting to fix it.

5.1.2 The Gap Between Reported and Actual Arrival

Ten of twelve reported successes were false. The mechanism is visible in the same records: median belief-to-truth drift in those trials was 6.25 m, against a median true error of 6.35 m. The two figures are close because they are essentially the same quantity: the robot was where its filter believed the goal to be, and the filter was wrong about where the robot was.

This is why the failure is unobservable from above. Every layer that could have detected the error was reading the same corrupted estimate. The planner planned a valid path from a false start; the controller followed it competently; the goal checker was satisfied; the language layer reported an arrival in correct English. Each component behaved exactly as it would have if the robot had been where it thought. There is no behavioural signature to detect, which is precisely why an independent measurement is required rather than a smarter monitor.

5.1.3 The Diagnostic Sequence

Three measurements localised the fault, and the order matters.

Reachability was eliminated first: all eight destinations lie in one connected traversable region with at least 1.09 m of clearance, and all returned zero cost in the live costmap. No reported failure was a goal the robot could not have reached.

The map was eliminated second, and this is the measurement that most constrains the explanation. A scan taken at the true pose put 100% of beams within 0.20 m of a mapped wall, with a mean of 0.047 m, and the heading sweep produced a single sharp minimum at the correct orientation – 0.047 m against 0.41 m to 0.76 m at every other tested heading. The environment is therefore richly informative and free of the geometric self-similarity that legitimately defeats a particle filter. A filter that diverges here is not being defeated by the world; it is misconfigured.

The step response then isolated the motion responsible. Standing still cost nothing; a single 90° rotation multiplied heading uncertainty by 9.3. Rotation, not translation and not time, was destroying the estimate.

5.1.4 The Correction That Was Backwards

The most instructive result in this thesis is the parameter sweep in Table 4.6, because the obvious inference from the step response was wrong.

The robot's casters are fixed rather than swivelling and scrub laterally during turns, so rotational odometry is genuinely the least trustworthy channel. The natural correction is to raise the rotational noise coefficient so the filter stops trusting it. That was done, to 0.25, and the system degraded: position error after a 90° turn rose to 0.469 m and heading uncertainty to 1.257 rad.

The sweep shows why. These coefficients scale a noise term that is itself proportional to the magnitude of the rotation, so a larger value disperses particles across an arc that widens faster than a single scan update can reconcile. The filter is not made appropriately humble; it is made unable to converge. Reducing the coefficient to 0.02 produced 0.108 m and 0.441 rad, monotonically better across the swept range.

The generalisable point is not the value. It is that a plausible causal story – the casters scrub, therefore admit more rotational noise – pointed in exactly the wrong direction, and that only measurement against ground truth distinguished the two. Had the system been evaluated on its own reports, the change would have appeared harmless, because the filter's reported confidence and its actual accuracy had already parted company.

5.1.5 The Audit of the Initial Run

The duration audit reduced the initial run's effective sample from 54 trials to 16, and this must be read carefully.

What survives intact is the headline finding. All 12 reported successes fall within the 16-trial group; the 38 short-duration trials reported none. The ratio $10/12$ is therefore computed entirely from trials that executed a navigation, and the causal attribution is strengthened rather than weakened, because median drift within that group is 6.25 m against 0.54 m in the group that did not navigate.

What does not survive is the denominator. Reporting "6% of 54 commands arrived" overstates the evidence, because 38 of those commands did not attempt the task. The honest

statement is that 16 navigation trials produced 12 reported successes and 2 real arrivals.

The audit also exposed the fourth fault. Navigation outcomes were written to a single slot carrying no identity of the goal that produced them, so a cancelled goal's verdict could be read as the next goal's, returning a result in seconds for a navigation that had not occurred. That this was found by a routine duration check, rather than by inspecting the code, is itself an argument for recording process metadata alongside outcomes.

5.1.6 The System After Correction

The corrected system arrived on 60 of 61 commands with a median true error of 0.231 m and no false positives among 50 reported successes. Median drift fell from 6.25 m to 0.108 m, and no trial required re-localisation or found the robot outside the map.

The residual discrepancy has inverted. The stack now reports fewer successes than actually occur: ten trials arrived to within 0.12 m to 0.39 m and were never confirmed. The instrumented case shows the robot inside its position tolerance with commanded linear velocity at zero and angular velocity of constant sign for 46 s – the signature of a recovery rotation rather than controlled steering. The progress checker in use measures translation only, so a robot that has satisfied position tolerance and is rotating to satisfy heading tolerance registers as making no progress, which triggers a recovery that also rotates and also fails the same test.

This is a materially better failure than the one it replaced. A false negative costs time and can be detected from the robot's own report; a false positive cannot be detected at all from within the system.

5.2 Comparison with Previous Studies

5.2.1 Agreement

The language-understanding results agree with the grounding literature. The finding that a constrained-output language model reliably maps varied phrasings onto a fixed action space is consistent with the premise of Code-as-Policies (Liang et al., 2023) and ProgPrompt (Singh et al., 2023), and with the affordance-conditioning approach of SayCan (Ahn et al., 2022). The hybrid design used here – rule fast-paths for audit queries, model parsing for everything else – is a narrowing of the same principle.

The perception results are unremarkable in the sense that matters: composing a class-agnostic segmenter (Kirillov et al., 2023) with an image–text scorer (Radford et al., 2021) produced open-vocabulary labels on a robot's live camera stream without a fixed label set, as that literature predicts.

The localisation pathologies are precisely those documented in the probabilistic-robotics literature (Thrun et al., 2005): a motion model whose noise is mis-scaled degrades convergence, and a diverged filter does not recover unaided. This thesis contributes a worked instance with the parameter values and the resulting errors, rather than a new phenomenon.

5.2.2 Disagreement

The disagreement concerns evaluation, and it is substantive. Where success is determined through the agent's pose estimate, the reported figure conflates task competence with localisation accuracy. This work shows the two coming apart by a factor of six on the same system, same map and same commands: a 75% reported success rate against a 12.5% arrival rate.

Stating that as a disagreement with the vision-and-language navigation literature would be a misreading, and the distinction matters enough to set out carefully. The measures that literature standardised — success rate, final distance to goal, and success weighted by path length (Anderson, Chang et al., 2018) — are well posed for the problem they were defined over. In the nav-graph formulation (Anderson, Wu et al., 2018) the agent selects nodes and the simulator moves it, so its position is exact by construction and the quantity this thesis measures is identically zero. There is nothing for such a benchmark to have missed.

The relationship is therefore one of scope rather than of contradiction. Those benchmarks hold the agent's self-knowledge fixed and vary its decision-making; this study holds decision-making close to fixed — language resolution was correct on 60 of 61 commands — and finds that self-knowledge is where the system fails. The two are complementary axes, and a system may score well on one while failing entirely on the other, which is precisely what was observed here.

Two bodies of work sit closer to the boundary. The continuous-environment reformulation (Krantz et al., 2020) removed the nav-graph and required the agent to execute rather than teleport, and reported that performance falls substantially once it must. That establishes the general point that execution is not free. It does not, however, reach the failure reported here: the agent still receives its position from the simulator, so a competent policy executing correctly cannot arrive somewhere it does not believe itself to be. And systems deployed on physical platforms, LM-Nav (Shah et al., 2023) among them, do face genuine localisation error — but report task success, which is the quantity that becomes unreliable when localisation degrades, rather than the gap between claimed and actual arrival.

The claim this thesis makes is accordingly narrow and, within that scope, strong. Where an estimated component mediates the success determination, the reported figure and the physical outcome must be measured separately, because they were found to differ by a factor of six on the same trials. Any benchmark that reports only the former is measuring the estimator and the policy together, without a means of telling which one moved.

This also bears on a second disagreement identified in Section 2.2: whether robustness should be sought by having the language layer reason about uncertainty, or by having the lower layers report it honestly. The results support the second position. A language layer reasoning over a corrupted pose has nothing to reason with – the trace in Section 4.7 shows the mechanism working only because the navigation layer exposed a covariance for the policy to read.

5.2.3 Comparison with the Original Project Objectives

The project brief specified validation with 50 or more language commands, and named a legged or larger wheeled platform. The command-count objective is met and exceeded, at 61 commands with per-trial ground-truth scoring rather than self-reported success. The platform objective is met by substitution: ROMR in simulation replaced the named hardware, for reasons given in Section 6.5.1.

5.3 Why the Results Are as They Are

Three factors account for the pattern observed.

Defaults transfer badly across platforms. Every localisation fault was a default or a plausible value carried from another robot: a 8 m sensor range against a 16 m scanner, a rotational noise coefficient an order of magnitude too large, and a hit-versus-noise weighting asserting that half of every scan is meaningless when 100% of beams fall within 0.20 m of a mapped wall. None was a sophisticated failure; each was a mismatch between a parameter and a physical fact that was measurable.

Layering hides the mismatch. The architecture that makes such a system tractable is the same architecture that conceals the fault. Each layer consumes its input as authoritative, so a corrupted pose propagates upward gaining apparent confirmation at every stage.

The measurement was cheap and was simply not taken. Ground truth in simulation costs one query per trial. The reason the discrepancy went undetected in the demonstration phase is not that detecting it was expensive but that the system was asked how it was doing and answered confidently.

5.4 Importance of the Findings

For evaluation practice. The distinction between reported and actual outcome should be reported as two numbers, not one, in any system where a learned or estimated component mediates the success determination. On the evidence here they differ by a factor of six.

For safety. A confidently wrong self-report is qualitatively worse than an acknowledged failure, because it cannot be caught downstream. As language interfaces are placed where a person acts on the robot's word, the false-positive rate becomes a safety property. The corrected system's inversion into false negatives is, on this argument, the right direction to fail.

For method. The parameter sweep result is the transferable contribution. A plausible mechanical account of a fault produced a correction of the wrong sign, and only independent measurement caught it. That is an argument for holding ground truth in the loop during debugging, not only during final evaluation.

5.5 Threats to Validity

Internal validity. The before-and-after comparison uses the same system as its own control. Four changes were applied between runs, so the contribution of each cannot be separated from the aggregate; the sweep in Table 4.6 isolates one of them, but the others are not individually attributed. The corrected run is a single run, so run-to-run variance is unquantified.

Construct validity. Arrival is operationalised as position within 0.5 m, ignoring final heading. A robot at the right place facing the wrong way counts as arrived. Table 4.3 shows the initial conclusion is insensitive to the threshold over 0.3–1.0 m.

External validity. One robot, one environment, eight destinations, one map, in simulation. The command suite is author-constructed and does not establish how non-expert users would phrase instructions. Nothing here establishes that the same faults or magnitudes would appear on other platforms, and no claim to that effect is made.

Statistical validity. No inferential test is applied, for the reason given in Section 3.7. The effect sizes are large and the comparison is within a single engineered artefact, but readers seeking a confidence interval on the corrected arrival rate will not find one here.

Simulation fidelity. Sensor noise, wheel slip and contact dynamics are modelled, not measured from hardware. The scrub behaviour that motivated the rotational-noise investigation is a property of the simulated contact model, and its correspondence to the physical robot is untested.

6 Conclusion

6.1 Summary of Main Findings

This thesis built a language-grounded navigation system for a differential-drive mobile robot – speech and text input, intent parsing with a constrained language model, open-vocabulary perception composing SAM with CLIP, a LangGraph command policy, and a ROS 2 Navigation 2 stack – and then built a second instrument to find out whether it did what it reported.

Four findings follow.

Language understanding was not the limiting factor. Sixty of sixty-one commands resolved to the intended destination, including phrasings sharing no lexical item with the registry key. In the initial run every command resolved correctly while the system arrived twice in sixteen trials.

Reported and actual arrival diverged by a factor of six. Twelve reported successes contained two real arrivals: 83% of the system's reported successes were false, with a median true error of 6.35 m.

The divergence was localisation, and it was configuration. Goals were reachable and the map was highly informative – 100% of beams within 0.20 m of a mapped wall at the true pose, with a unique heading minimum. Rotation alone degraded the estimate, multiplying heading uncertainty by 9.3 in one 90° turn. Three parameter faults and one outcome-bookkeeping fault accounted for the behaviour.

The identical protocol showed the divergence removed. After correction, 60 of 61 commands arrived, median true error 0.231 m, zero false positives among 50 reported successes, and median belief-to-truth drift of 0.108 m against 6.25 m.

6.2 Key Conclusions

A system's report about itself is not evidence for what it reports. This is the conclusion on which the others rest. Every layer above localisation confirmed an arrival that had not occurred, not through any defect of its own but because each consumed the same corrupted estimate as authoritative.

End-to-end success rate is misleading for composed systems. The same system produced a 100% language score and a 12.5% arrival rate. A single end-to-end figure would have directed effort at the component that was already working.

A plausible causal account is not a substitute for measurement. The fixed casters do scrub, and admitting more rotational noise is the obvious remedy. It was the wrong sign, and the system got worse. Only a sweep scored against ground truth established the direction.

Failing loudly is better than failing silently. The corrected system now under-reports: ten arrivals within 0.39 m went unconfirmed. That is a defect, but a detectable one, and therefore

preferable to the undetectable failure it replaced.

6.3 Contribution of the Study

A ground-truth evaluation protocol for language-grounded navigation. A harness that scores arrival from the simulator's own state, sharing no sensor, filter or frame with the robot's estimate, and reports reported success and actual arrival as separate quantities with the false-positive rate between them.

A quantified instance of confidently wrong self-report. A measured 83% false-positive rate among reported successes, with the causal attribution carried through to parameter level rather than left as a limitation.

A worked negative result and its resolution. The complete arc from measured failure, through elimination of reachability and map quality, through isolation of the responsible motion, to a parameter sweep that reversed an intuitive correction, to a re-run of the identical protocol.

An implemented architecture in which verification is a first-class node. A command policy that waits for the real navigation outcome rather than treating dispatch as success, and declines to confirm arrival when the pose covariance does not support it.

A reproducible artefact. The implementation, command suites, per-trial records and analysis scripts behind every figure reported here.

6.4 Practical Recommendations

1. **Hold ground truth in the loop during development, not only at evaluation.** It cost one query per trial and it caught a correction of the wrong sign.

2. **Report reported and actual outcomes as two numbers.** Wherever an estimated component mediates success determination, publish both and the false-positive rate between them.

3. **Audit trial duration before trusting a rate.** A routine check against expected task duration exposed a fault that had silently corrupted a majority of an earlier run.

4. **Treat inherited parameters as claims about the hardware.** Every localisation fault here was a value that was true of some other robot. Each should be checked against a measurable physical fact.

5. **Make lower layers expose uncertainty rather than verdicts.** The refusal-to-confirm mechanism worked only because the navigation layer published a covariance for the policy to read.

6. **Match the progress checker to the motion the robot must make.** A translation-only progress test penalises the terminal rotation that a pose goal requires.

6.5 Limitations of the Study

6.5.1 Hardware Deployment Was Not Achieved

The physical ROMR exists and deployment was attempted under supervision. The on-board Jetson Nano could not host the segmentation and image–text models, and the perception pipeline could not be run on-robot. Every quantitative result in this thesis is therefore from simulation, and the correspondence between the simulated contact dynamics – including the caster scrub central to the localisation analysis – and the physical platform is untested.

6.5.2 Remaining Defects

The corrected system under-reports arrival: ten of sixty arrivals were not confirmed, each consuming the full time budget and a retry. The cause is identified in Section 4.5.2 but the fix is not validated here.

Localisation is seeded rather than solved globally. The filter is given an initial pose and has divergence recovery disabled, a choice justified by the map's informativeness but one that means a genuinely kidnapped robot would require manual re-seeding.

6.5.3 Scope and Statistical Limitations

One robot, one environment, eight destinations, one map. The command suite is author-constructed and no user study was conducted, so nothing here establishes how non-expert speakers would phrase instructions. The before-and-after comparison changed four things at once and rests on single runs, so per-fault contributions and run-to-run variance are unquantified. No inferential statistics are reported.

6.5.4 Perception Is Peripheral to the Measured Result

Perception describes the scene and answers questions about it, but navigation goals resolve from a recorded registry rather than from visual search. Perception accuracy therefore does not enter the arrival measurement, and the open-vocabulary results reported here are demonstrated rather than systematically evaluated.

6.6 Suggestions for Future Research

Replicate the protocol on hardware. The natural next step is a motion capture or fiducial ground-truth channel on the physical robot, replicating the same 61-command protocol. This would test whether the simulated scrub behaviour and the parameter values derived from it transfer.

Characterise the compute boundary. The Jetson Nano result is a data point, not a study. Profiling which components of a vision–language–action pipeline can run on constrained

edge hardware, at what latency, and which must be offloaded, is a well-posed question that this project ran into rather than answered.

Close the loop on verification. The system currently detects that it is uncertain and declines to confirm. A stronger system would act on that: re-localise deliberately, seek a distinctive viewpoint, or ask the operator. Using the perception stack to verify arrival visually – checking that the scene matches what is expected at the destination – would provide a second independent check that does not depend on the particle filter.

Ground goals in perception rather than a registry. Resolving destinations by embedding similarity against a persistent open-vocabulary spatial representation would remove the recorded registry and make perception load-bearing for navigation, at which point its accuracy would enter the arrival measurement.

Extend the false-positive analysis to manipulation. "Did the robot actually place the part?" has the same structure as "did it actually arrive?", and in a shared human–robot workspace the consequences of a confidently wrong self-report are more immediate.

Quantify per-fault contributions. An ablation applying each of the four corrections independently would establish how much of the improvement each accounts for, which this study cannot separate.

6.7 Closing Remarks

The system described in this thesis demonstrated well before it was measured. It parsed instructions correctly, planned sensible paths, moved competently, and reported its successes in fluent language. Measured against a source of truth it did not control, it was arriving at 17% of the places it claimed.

The gap was not in the part of the system that looked hardest. It was in a handful of parameters describing a sensor and a drivetrain, inherited from a robot that was not this one. Finding them required only that the question be asked from outside the system – and the most useful result of the project came not from building the robot, but from building the instrument that could contradict it.

Bibliography

Ahn, M., Brohan, A., Brown, N., Chebotar, Y., Cortes, O., David, B., Finn, C., Fu, C., Gopalakrishnan, K., Hausman, K., et al. (2022). Do as i can, not as i say: Grounding language in robotic affordances. *Proceedings of the 6th Conference on Robot Learning (CoRL)*.

Anderson, P., Chang, A., Chaplot, D. S., Dosovitskiy, A., Gupta, S., Koltun, V., Kosecka, J., Malik, J., Mottaghi, R., Savva, M., & Zamir, A. R. (2018). On evaluation of embodied navigation agents. *arXiv preprint arXiv:1807.06757*. https://arxiv.org/abs/1807.06757

Anderson, P., Wu, Q., Teney, D., Bruce, J., Johnson, M., Sünderhauf, N., Reid, I., Gould, S., & van den Hengel, A. (2018). Vision-and-language navigation: Interpreting visually-grounded navigation instructions in real environments. *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*.

Chen, B., Xia, F., Ichter, B., Rao, K., Gopalakrishnan, K., Ryoo, M. S., Stone, A., & Kappler, D. (2023). Open-vocabulary queryable scene representations for real world planning. *Proceedings of the IEEE International Conference on Robotics and Automation (ICRA)*.

Es, S., James, J., Espinosa-Anke, L., & Schockaert, S. (2024). RAGAs: Automated evaluation of retrieval augmented generation. *Proceedings of the 18th Conference of the European Chapter of the Association for Computational Linguistics (EACL): System Demonstrations*.

Fox, D., Burgard, W., & Thrun, S. (1997). The dynamic window approach to collision avoidance. *IEEE Robotics & Automation Magazine*, *4*(1), 23–33.

Gadre, S. Y., Wortsman, M., Ilharco, G., Schmidt, L., & Song, S. (2023). CoWs on PASTURE: Baselines and benchmarks for language-driven zero-shot object navigation. *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*.

Huang, C., Mees, O., Zeng, A., & Burgard, W. (2023). Visual language maps for robot navigation. *Proceedings of the IEEE International Conference on Robotics and Automation (ICRA)*.

Kirillov, A., Mintun, E., Ravi, N., Mao, H., Rolland, C., Gustafson, L., Xiao, T., Whitehead, S., Berg, A. C., Lo, W. Y., Dollár, P., & Girshick, R. (2023). Segment anything. *arXiv preprint arXiv:2304.02643*. https://arxiv.org/abs/2304.02643

Krantz, J., Wijmans, E., Majumdar, A., Batra, D., & Lee, S. (2020). Beyond the nav-graph: Vision-and-language navigation in continuous environments. *Proceedings of the European Conference on Computer Vision (ECCV)*. https://arxiv.org/abs/2004.02857

Lewis, P., Perez, E., Piktus, A., Petroni, F., Karpukhin, V., Goyal, N., Küttler, H., Lewis, M., Yih, W.-t., Rocktäschel, T., Riedel, S., & Kiela, D. (2020). Retrieval-augmented generation for knowledge-intensive NLP tasks. *Advances in Neural Information Processing Systems (NeurIPS)*.

Li, J., Li, D., Savarese, S., & Hoi, S. C. (2023). Blip-2: Bootstrapped language–image pre-training with frozen image encoders and large language models. *arXiv preprint arXiv:2301.12597*. https://arxiv.org/abs/2301.12597

Liang, J., Huang, W., Xia, F., Xu, P., Hausman, K., Ichter, B., Florence, P., & Zeng, A. (2023). Code as policies: Language model programs for embodied control. *2023 IEEE International Conference on Robotics and Automation (ICRA)*, 9493–9500. https://doi.org/10.1109/ICRA48891.2023.10160891

Macenski, S., Martín, F., White, R., & Ginés Clavero, J. (2020). The marathon 2: A navigation system. *Proceedings of the IEEE/RSJ International Conference on Intelligent Robots and Systems (IROS)*.

Nwankwo, L., Ellensohn, B., Özdenizci, O., & Rueckert, E. (2025). Reli: A language-agnostic approach to human–robot interaction. *arXiv preprint arXiv:2505.01862*. https://arxiv.org/abs/2505.01862

Nwankwo, L., & Rueckert, E. (2024). The conversation is the command: Interacting with real-world autonomous robots through natural language. *Companion of the 2024 ACM/IEEE International Conference on Human–Robot Interaction*, 808–812. https://doi.org/10.1145/3610978.3640352

Nwankwo, L., Fritze, C., Bartsch, K., & Rückert, E. (2023). ROMR: A ros-based open-source mobile robot. *HardwareX*, *15*, e00426.

Radford, A., Kim, J. W., Hallacy, C., Ramesh, A., Goh, G., Agarwal, S., Sastry, G., Askell, A., Mishkin, P., Clark, J., Krueger, G., & Sutskever, I. (2021). Learning transferable visual models from natural language supervision. *Proceedings of the 38th International Conference on Machine Learning*, 8748–8763.

Shah, D., Osiński, B., Ichter, B., & Levine, S. (2023). LM-Nav: Robotic navigation with large pre-trained models of language, vision, and action. *Proceedings of the 6th Conference on Robot Learning (CoRL)*.

Singh, I., Blukis, V., Mousavian, A., Goyal, A., Xu, D., Tremblay, J., Fox, D., Thomason, J., & Garg, A. (2023). Progprompt: Generating situated robot task plans using large language models. *2023 IEEE International Conference on Robotics and Automation (ICRA)*, 11523–11530. https://doi.org/10.1109/ICRA48891.2023.10161444

Thrun, S., Burgard, W., & Fox, D. (2005). *Probabilistic robotics*. MIT Press.

A Reproducing the Evaluation

Chapter 4 reports figures obtained from a fixed command suite run against a stack brought up in a particular order. Neither is recoverable from the source code alone: the suite is data, and the bring-up encodes decisions about the host that are invisible in the launch files. Both are given here so that the results can be reproduced rather than accepted.

A.1 The Command Suite

Table A.1 lists all 61 phrasings, grouped by the destination each was intended to reach. The suite is constructed rather than sampled from users, and is designed to exercise lexical variation: direct naming, polite and indirect forms, verb variation, and synonyms that share no token with the registry key. Twelve phrasings name their destination only by a synonym — *lounge*, *sitting room* and *living area* for `parlour`; *car park* and *parking space* for `garage`; *reception* and *lobby* for `waiting_area`; *docking station* for `home` — and these are the cases that test grounding rather than string matching.

The suite is defined in `scripts/validate_commands.py` and this table is generated from it, so the two cannot drift apart.

Table A.1. The 61 natural-language commands, by intended destination.

#	Destination	Phrasing
1	`kitchen`	go to the kitchen
2		kitchen
3		please head to the kitchen
4		can you drive to the kitchen
5		move to the kitchen now
6		I need you in the kitchen
7		navigate to kitchen
8		take yourself to the cooking area
9	`parlour`	go to the parlour
10		go to the living room
11		head to the lounge
12		drive to the sitting room
13		please go to the parlour
14		move to the living area
15		navigate to the lounge
16	`waiting_area`	go to the waiting area
17		drive to the visitors waiting area

Table A.1 continued

#	Destination	Phrasing
18		head to reception
19		please go to the lobby
20		take the robot to the waiting area
21		navigate to the visitors area
22		wait in the waiting area
23	`garage`	go to the garage
24		drive into the garage
25		please move to the garage
26		head over to the garage
27		navigate to the garage
28		take the robot to the car park
29		go park in the parking space
30		could you go to the garage
31	`store_area`	go to the store area
32		head to the storage
33		drive to the store
34		please go to the store area
35		move to storage
36		navigate to the storage area
37		go to the store room
38		drive over to the storeroom
39	`dining_room`	go to the dining room
40		head to the dining area
41		please drive to the dining room
42		move to the dining room
43		navigate to the dining area
44		take me to the dinner table
45		go and wait in the dining room
46		head for the dining area please
47	`master_bedroom`	go to the master bedroom
48		head to the bedroom
49		please go to the master bedroom
50		drive to the bedroom
51		navigate to the main bedroom
52		move to the master room
53		go up to the master bedroom

Table A.1 continued

#	Destination	Phrasing
54		please make your way to the bedroom
55	`home`	go home
56		return home
57		head back home
58		please go to home base
59		navigate home
60		take the robot to the docking station
61		go back to base

A.2 Bringing the Stack Up

The system runs as five processes started in order, each holding a terminal. Order matters: every stage waits on the one before it, and starting the navigation stack before the simulator produces a set of nodes that answer on their topics while describing a robot that does not exist.

1. **Clear any previous session.**

   ```
   scripts/clean.sh --all
   ```

   This is not housekeeping. A navigation container left bound to a simulator that has since exited continues to answer on its topics, and additional `robot_state_publisher` processes feed the simulator an older robot description — symptoms that present as application faults and are not. The `--all` form additionally frees the two web ports.

2. **Simulator and robot.**

   ```
   scripts/sim.sh
   ```

   Wrapped rather than invoked directly because the environment script strips the sandbox variables an editor-hosted terminal injects, which otherwise kill the Gazebo client, and because this host has no multicast route: the middleware is configured for unicast loopback on a non-default domain.

3. **Navigation stack.**

   ```
   scripts/nav2_rviz.sh
   ```

   Roughly 40 s to activate. The visualiser is included because the initial pose can be set by hand from it, and because the inflated costmap the planner actually searches is otherwise not observable.

4. **Backend** (`scripts/backend.sh`) and **interface** (`scripts/frontend.sh`), on ports 8000 and 3000.

Seeding the Localisation

The particle filter is configured with `set_initial_pose: false` and therefore publishes no `map` → `odom` transform until it is given a pose. Every goal fails to plan before that point. Either

```
scripts/seed.sh
```

which seeds from the simulator's own pose, or the visualiser's pose-estimate tool. The robot must then *move*: a particle filter updates only once it has travelled past its update thresholds, so a stationary robot retains whatever spread it was seeded with. The seeding script rotates for this reason.

A.3 Running the Evaluation

With the stack up:

```
python3 scripts/validate_commands.py --mode graph \
        --out results/validation.csv
```

Approximately one hour for the full suite, since each trial waits for the real navigation outcome. The harness writes one row per trial, containing the phrasing, the place resolved, the outcome reported, the three distances of Equations (3.3)–(3.5), the filter's covariance at the moment the outcome was declared, and the trial duration.

The figures of Chapter 4 are regenerated from those rows by `scripts/plot_results.py`, and the particle-set figures of Chapter 3 by `scripts/particles.sh` and `scripts/particles_approx.sh`. No figure in this thesis was drawn by hand from remembered numbers.

A.4 A Note on Reading the Output

Trial duration should be inspected before any rate derived from the run is believed. A navigation in this environment takes tens of seconds; a trial returning in a few is reporting an outcome that the navigation it purports to describe did not produce. Applying that check to an earlier run of this suite reduced its effective sample from 54 trials to 16 and exposed a defect in how outcomes were attributed to goals, as recorded in Section 4.1.